

COCO: A Cohesiveness-aware Learning Framework for Community Search over Temporal Graphs

Chengyang Luo
Zhejiang University
luocy1017@zju.edu.cn

Chuanyu Liu
Zhejiang University
liucy219@zju.edu.cn

Qing Liu
Zhejiang University
qingliucs@zju.edu.cn

Jianliang Xu
Hong Kong Baptist University
xujl@comp.hkbu.edu.hk

Yunjun Gao
Zhejiang University
gaoyj@zju.edu.cn

ABSTRACT

We study time-interval-based community search over temporal graphs, aiming to identify a connected cohesive subgraph containing query vertices within a given interval. Existing work suffers from two main limitations: (1) *information loss*, as it ignores the interaction frequency between vertices, a crucial factor for identifying high-quality communities. (2) *efficiency bottleneck*, as it either incurs high search latency from expensive online computation or faces a combinatorial explosion in indexing time and storage when precomputing coreness values across all possible time intervals. To address these issues, we propose COCO, a cohesiveness-aware learning framework. COCO introduces a new adaptive query processing mechanism that uses interval-specific coreness predictions to construct structural supervision and adaptation context. A *hierarchical model index* efficiently predicts coreness over arbitrary query intervals through coarse-to-fine refinement, providing structural guidance for community search. An *adaptive temporal GNN* uses these predictions to construct training samples and query context, jointly capturing structural cohesiveness and interaction frequency through lightweight adapter updates. Experiments on eight real-world datasets show that COCO outperforms state-of-the-art methods in community quality and is at least 10× faster than the baselines, demonstrating its effectiveness and efficiency.

PVLDB Reference Format:

Chengyang Luo, Chuanyu Liu, Qing Liu, Jianliang Xu, and Yunjun Gao. COCO: A Cohesiveness-aware Learning Framework for Community Search over Temporal Graphs. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/ZJU-DAILY/COCO>.

1 INTRODUCTION

Community search is an important tool in graph analysis, aiming to identify cohesive subgraphs that contain given query vertices [5, 10, 26, 40]. Prior research has extensively investigated community search on directed graphs [2, 6, 21], bipartite graphs [37, 38], attributed graphs [22, 41], and multilayer graphs [1, 24, 33].

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX

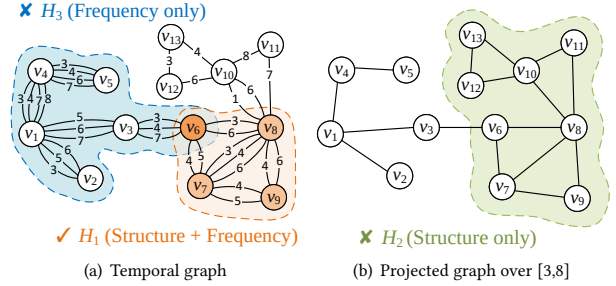


Figure 1: A motivating example

In real-world applications, interactions between entities evolve over time. For example, timestamped comments, likes, and shares in social networks record changing relationships. This temporal information helps explain user behavior and community structure. Community search over temporal graphs has thus attracted considerable attention [17–20, 43, 46]. For example, QTCS [20] is a state-of-the-art temporal community search method that uses a query-centered k -core variant, the β -temporal proximity core. In this core, each vertex’s query-biased temporal degree (the sum of its neighbors’ temporal-proximity scores to the query vertex) is at least β . The returned community retains a core structure while remaining temporally close to the query vertex. For community search over a specified time interval, [43] proposes the temporal k -core method. Given a temporal graph, a query vertex, a time interval, and an integer k , it returns a connected k -core containing the query vertex in a projected graph. This projected graph retains edges within the specified interval and merges multiple edges between the same vertex pair into one.

Figure 1(a) shows a temporal graph whose edge timestamps record when interactions occurred. Given the query vertex v_6 , the time interval $[3, 8]$, and $k = 2$, the method in [43] identifies the temporal 2-core $H_2 = \{v_6, \dots, v_{13}\}$ in the corresponding projected graph, as shown in Figure 1(b). QTCS applied to the temporal graph over $[3, 8]$ with the same query vertex v_6 also returns H_2 .

Limitations. When performing time-interval-based community search, these methods exhibit two major limitations.

L1. Information Loss. In Figure 1, the temporal k -core method [43] and QTCS [20] both return the vertex set $H_2 = \{v_6, \dots, v_{13}\}$ for query vertex v_6 . Within $[3, 8]$, every vertex pair involving v_{10} , v_{11} , v_{12} , or v_{13} interacts only once. Both methods emphasize structural cohesiveness, while QTCS also considers temporal proximity to the query vertex. Both nevertheless include these less active vertices. Interaction frequency (the number of distinct temporal edges

between two vertices) distinguishes single interactions from repeated ones and measures connection strength. However, using only interaction frequency can produce a community with a weak structure. Every edge in $H_3 = \{v_1, \dots, v_6\}$ occurs at least three times, but H_3 is a tree. The query vertex v_6 is connected to the remaining vertices only through (v_3, v_6) . In contrast, the community $H_1 = \{v_6, v_7, v_8, v_9\}$ is better than H_2 and H_3 because it considers both structural cohesiveness and interaction frequency.

L2. Efficiency Bottleneck. Both the method in [43] and QTCS [20] have serious efficiency limitations. Performing online core decomposition for each query interval is computationally expensive and leads to high search latency on large-scale graphs. For example, on graphs with millions of vertices, online k -core decomposition may take around 500 seconds, while QTCS, which processes each query from scratch, may take several hundred seconds (see Section 6.2 for details). Although indexing can accelerate temporal k -core queries, it requires precomputing and storing results for all possible k values and time intervals. Since a temporal graph with $t_{\max}$ timestamps contains $O(t_{\max}^2)$ continuous intervals, this incurs prohibitive preprocessing and storage costs on large graphs. For a graph with two million vertices, the index can exceed 1 GB; on larger graphs, construction can take over 24 hours (see Section 6.2).

To address these limitations, we revisit and formalize the *Time-Interval-based Community Search (TICS)* problem. Specifically, given a temporal graph $\mathcal{G}$, a time interval $[t_s, t_e]$, and a set of query vertices V_Q , the goal of TICS is to identify a community containing V_Q within $[t_s, t_e]$.

Challenges. The TICS problem presents two main challenges.

C1. How to identify high-quality communities? High-quality temporal communities should exhibit both cohesive structural topology and frequent temporal interactions. However, integrating these two heterogeneous signals is intrinsically difficult. Existing structure-based cohesive subgraphs cannot effectively measure interaction frequency, while purely frequency-based metrics often overlook the overall specific community structures.

C2. How to address TICS efficiently? The time interval of each TICS query may differ, and therefore the graph topology associated with each query can also be different. Processing every TICS query from scratch is computationally expensive, especially on large-scale temporal graphs. Although preprocessing or indexing may help, such methods must handle the large number of possible graph structures across all time intervals, further increasing costs.

Our Solutions. To overcome these challenges, we propose **COCO**, a novel **CO**hesiveness-aware learning framework for **CO**mmunity search over temporal graphs. COCO introduces a new adaptive query processing mechanism that uses interval-specific coreness predictions to construct structural supervision and adaptation context. In this mechanism, coreness serves as an explicit structural prior during model training and query-specific adaptation, thereby tightly integrating the classical k -core model with graph neural networks (GNNs). COCO offers two key advantages: (1) it identifies communities with both structural cohesiveness and frequent interactions; and (2) it efficiently processes arbitrary-interval queries without exact core decomposition or full model retraining.

Specifically, COCO implements this mechanism through two components. (1) The **Hierarchical Model Index (HM-Index)** pro-

vides interval-specific coreness predictions for constructing structural supervision and adaptation context (**for C2**). It recursively partitions the temporal span into nested intervals, each represented by a tree node containing a lightweight prediction model. For a query interval, HM-Index traverses the tree from the root, with each visited child model refining its parent’s prediction using more local interval information. This coarse-to-fine refinement predicts coreness over arbitrary query intervals without exact online core decomposition or exhaustive computation over all $O(t_{\max}^2)$ possible intervals. (2) The **Adaptive Temporal GNN (AT-GNN)** learns vertex representations that capture structural cohesiveness and interaction frequency (**for C1**). Its temporal encoder captures interaction patterns, while predicted coreness guides training sample construction and supplies query and neighborhood statistics to lightweight query-aware adapters in each encoder layer. Only these adapters are fine-tuned for each query, using structural context to adjust vertex representations efficiently (**for C2**).

COCO operates through a three-phase pipeline. First, in the pre-training phase, COCO trains coreness prediction models on different time intervals and then organizes these models into an HM-Index, and trains AT-GNN in an unsupervised manner, enabling it to learn common community patterns across time intervals. In the fine-tuning phase, COCO predicts the coreness of query vertices and their neighbors by traversing the HM-Index, and uses these coreness values along with the query context to adjust the adapter module parameters of AT-GNN. In the searching phase, COCO uses vertex embeddings from AT-GNN to iteratively select vertices most similar to the query vertices, forming the final community.

Contributions. Overall, our main contributions are as follows.

- We propose COCO, a cohesiveness-aware learning framework for TICS. Its new adaptive query processing mechanism uses interval-specific coreness predictions to construct structural supervision and adaptation context, enabling efficient search for high-quality communities over arbitrary time intervals.
- We design the *Hierarchical Model Index* (HM-Index), which organizes lightweight models in a hierarchical tree. Its coarse-to-fine refinement predicts coreness over arbitrary intervals without computing coreness for all $O(t_{\max}^2)$ intervals, reducing preprocessing time and storage costs.
- We develop the *Adaptive Temporal GNN* (AT-GNN) to jointly capture graph structure and interaction frequency. Its query-aware adapters enable fast adaptation to different query structures without full model retraining.
- We conduct extensive experimental studies on eight real-world graphs. The results validate the superior effectiveness and efficiency of our proposed COCO framework.

2 RELATED WORK

We review classic temporal community search and learning-based community search. Table 1 compares representative temporal community search methods. COCO is the only method that jointly models cohesive subgraph structure and pairwise interaction frequency while natively supporting TICS without requiring full retraining.

Classic community search over temporal graphs. Most active community search [17] uses a (k, t) -active core, while significant engagement community search [46] requires a connected k -core

Table 1: Comparison of temporal community search methods

Method	Underlying Model	Native TICS Support	Structural Cohesiveness Modeling	Interaction Frequency Modeling	No Full Recomputation/Retraining for TICS
Temporal k -core [43]	k -core	✓	✓	✗	✓
(k, t) -active core [17]	k -core	✗	✓	✗	✓
SECS [46]	k -core	✗	✓	✗	✗
Persistent core [18, 19]	k -core	✗	✓	✗	✗
QTCS [20]	k -core + TPPR	✗	✓	✗	✗
CS-TGN [9]	GNN	✗	✗	✗	✗
UTCS [45]	node2vec	✗	✗	✗	✗
PK-GCN [12]	GNN	✗	✓	✗	✗
COCO (our)	k -core + GNN	✓	✓	✓	✓

and maximizes the engagement of the query vertex. Persistent community search [18, 19] operates on snapshot sequences and seeks k -core communities that remain cohesive over time. QTCS [20] performs query-centered temporal community search. It derives Time-Constrained Personalized PageRank (TPPR) scores from chronological walks and defines a vertex’s query-biased temporal degree as the sum of its neighbors’ TPPR scores. QTCS maximizes the minimum query-biased temporal degree in a connected β -temporal proximity core, favoring neighbors temporally close to the query vertex. For a specified time interval, temporal k -core component search [43] requires every vertex to have a projected degree of at least k . These methods do not directly measure the interaction frequency between vertex pairs. In contrast, COCO searches over a specified interval while jointly considering structural cohesiveness and interaction frequency within the returned community. Other temporal cohesive subgraph models include span core [7, 8], periodic clique [28], periodic core [30], temporal k -core [42, 44, 47], and bursting core [29]. These models do not specifically target query-vertex-centered community search.

Learning-based methods for community search. Learning-based community search includes supervised methods using ground-truth communities [4, 11, 36], semi-supervised methods using partial labels [16, 32], and unsupervised methods using contrastive objectives or pseudo-labels [35, 39]. Most target static graphs and do not directly address TICS over arbitrary query intervals.

CS-TGN constructs query-dependent representations from local and global snapshot structures [9], while UTCS pre-trains temporal dynamics and subgraph knowledge [45]; PK-GCN applies a GNN to persistent k -core community search [12]. CS-TGN and UTCS do not explicitly model community cohesiveness, whereas PK-GCN is limited to persistent k -core structures. COCO instead uses an adaptive query processing mechanism: HM-Index predicts coreness for arbitrary query intervals, and AT-GNN uses these predictions to construct training samples and query context while capturing temporal interaction patterns. It avoids materializing coreness for all $O(t_{\max}^2)$ intervals and full GNN retraining per query.

3 PRELIMINARY

A temporal graph is represented as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ and $\mathcal{E}$ denote the sets of vertices and temporal edges, respectively. Each edge $e \in \mathcal{E}$ is a triplet (u, v, t) , representing an interaction between vertices u and v at timestamp t . The entire time interval of $\mathcal{G}$ is $\mathcal{T} = [1, t_{\max}]$. For a given time interval $[t_s, t_e] \subseteq \mathcal{T}$, the temporal graph $\mathcal{G}$ over $[t_s, t_e]$ is denoted by $\mathcal{G}_{[t_s, t_e]} = (\mathcal{V}_{[t_s, t_e]}, \mathcal{E}_{[t_s, t_e]})$, where $\mathcal{V}_{[t_s, t_e]} = \mathcal{V}$ and $\mathcal{E}_{[t_s, t_e]} = \{(u, v, t) \mid (u, v, t) \in \mathcal{E}, t \in [t_s, t_e]\}$.

The projected graph of $\mathcal{G}$ over $[t_s, t_e]$ is denoted by $\mathcal{G}_{pro[t_s, t_e]} = (\mathcal{V}_{pro[t_s, t_e]}, \mathcal{E}_{pro[t_s, t_e]})$. Here, $\mathcal{V}_{pro[t_s, t_e]} = \mathcal{V}$ and $\mathcal{E}_{pro[t_s, t_e]} = \{(u, v) \mid (u, v, t) \in \mathcal{E}, t \in [t_s, t_e]\}$. We use $N(v)$ to denote the neighbors of vertex v in $\mathcal{G}$. For the neighbors of v over a specific time interval $[t_s, t_e]$, we denote this set as $N_{[t_s, t_e]}(v)$. The degree of v at a particular timestamp $t \in \mathcal{T}$ is denoted by $d_t(v)$. Additionally, the degree of v over the time interval $[t_s, t_e]$ is $d_{[t_s, t_e]}(v) = |N_{[t_s, t_e]}(v)|$. Based on the above notations, we formally define our problem. For the query interval T_Q , we write $N_{T_Q}(v) = \{u \in \mathcal{V} \mid \exists t \in T_Q : (u, v, t) \in \mathcal{E}\}$. For a vertex set S , $N_{T_Q}(S) = (\bigcup_{v \in S} N_{T_Q}(v)) \setminus S$.

Problem 1. (Time-Interval-based Community Search (TICS)). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a time interval $T_Q = [t_s, t_e]$, and a set of query vertices $V_Q \subseteq \mathcal{V}$, the time-interval-based community search aims to find a vertex set C satisfying:

- (1) $V_Q \subseteq C$;
- (2) The subgraph induced by C in $\mathcal{G}_{[t_s, t_e]}$ is connected;
- (3) Vertices in C are densely intra-connected in $\mathcal{G}_{[t_s, t_e]}$.

Condition (3) requires both structural cohesiveness and frequent internal interactions. Temporal k -core constrains minimum degree in the projected graph, while QTCS [20] maximizes the minimum query-biased temporal degree based on temporal proximity to the query vertex. We next introduce key concepts.

Definition 3.1. (Temporal k -core [43]). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and a time interval $[t_s, t_e]$, let $\mathcal{G}_{pro[t_s, t_e]}$ be the corresponding projected graph. The temporal k -core over $[t_s, t_e]$ is the maximum subgraph $\mathcal{H} \subseteq \mathcal{G}_{pro[t_s, t_e]}$ such that each vertex in $\mathcal{H}$ has at least k neighbors.

Definition 3.2. (Coreness). Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and a time interval $[t_s, t_e]$, the coreness of a vertex $v \in \mathcal{V}$ over $[t_s, t_e]$, denoted by $c_{[t_s, t_e]}(v)$, is the largest k value such that v belongs to a temporal k -core in $\mathcal{G}_{pro[t_s, t_e]}$.

Example 1. Take Figure 1 as an example. Let the time interval be $[3, 8]$. The vertices v_6 through v_{13} form a temporal 2-core. All these vertices have coreness 2, i.e., $c_{[3, 8]}(v_6) = \dots = c_{[3, 8]}(v_{13}) = 2$.

Graph Neural Networks. GNNs [34] follow a neighborhood aggregation mechanism. This process can be expressed as:

$$\mathbf{h}_v^{(l)} = M\left(\mathbf{h}_v^{(l-1)}, \text{AGG}\left\{\mathbf{h}_u^{(l-1)} : u \in N(v)\right\}\right) \quad (1)$$

Here, $\mathbf{h}_v^{(l)}$ is the representation of vertex v in layer l , AGG is the aggregation function that combines messages from neighbors, and M is the message propagation function that updates the representation of the vertex using the aggregated representations and its own representation from the previous layer.

4 COCO FRAMEWORK

To address the TICS problem, we present COCO, a cohesiveness-aware learning framework. COCO introduces an adaptive query processing mechanism that uses interval-specific coreness predictions to construct structural supervision and adaptation context. Compared to existing learning-based community search approaches, COCO is capable of identifying communities that demonstrate strong cohesiveness and can flexibly adapt to different graph structures across various queries.

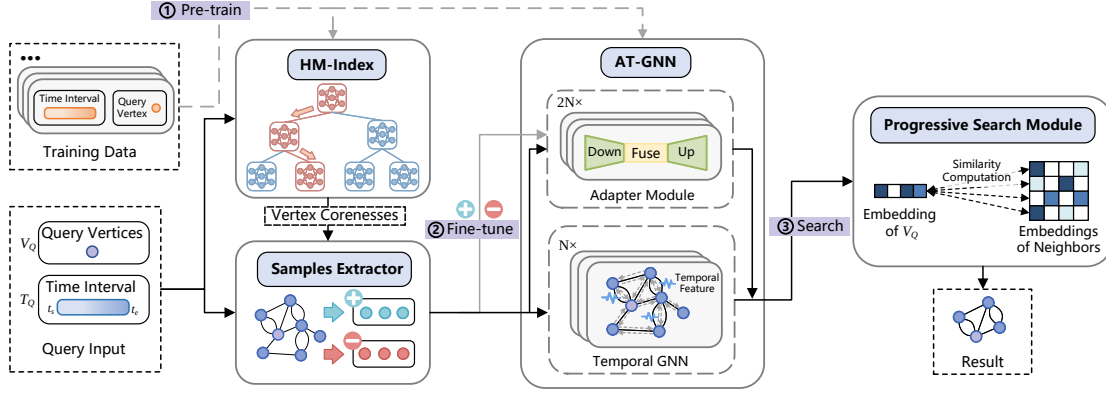


Figure 2: The design of COCO framework

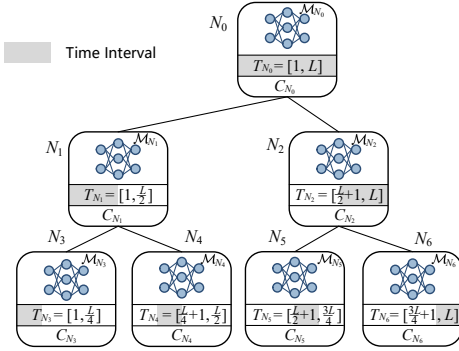


Figure 3: Architecture of HM-Index

Figure 2 shows COCO’s three-stage pipeline: pre-training, fine-tuning, and searching. (1) In pre-training, the HM-Index learns to predict vertex coreness over time, while the AT-GNN captures common temporal community patterns. (2) During fine-tuning, the HM-Index predicts coreness for query vertices V_Q and their neighbors in the time interval T_Q . A sample extractor then creates positive and negative examples based on these coreness values and the query context, used to fine-tune the AT-GNN’s adapter module. (3) In the search phase, AT-GNN generates vertex embeddings. The progressive search module iteratively selects vertices most similar to V_Q , adding them to the community until constraints are met. Some proofs appear in the *technical report* [23].

This section defines the architectures and inference procedures of HM-Index and AT-GNN, while Section 5 describes how they are constructed, trained, and used. The interval-partition rule defines HM-Index, and Algorithm 3 applies it during construction.

4.1 Hierarchical Model Index

To integrate coreness with GNNs, we need to compute coreness for arbitrary query intervals T_Q , since different intervals may produce dramatically different graph topologies. Computing coreness dynamically for each query is accurate but costly, while precomputing coreness for all $O(t_{\max}^2)$ possible subintervals requires excessive storage and preprocessing time [44]. This presents a fundamental trade-off between efficiency and flexibility.

We propose a learning-based method that predicts coreness instead of computing it directly. Our approach divides the entire time span into smaller intervals and uses lightweight regression models to predict coreness within each. These models are organized into a

Algorithm 1: Coreness Prediction Algorithm

Input: A temporal graph $\mathcal{G}$, a vertex v , a query time interval $[t_s, t_e]$, HM-Index root node N_{root}
Output: Predicted coreness $\hat{c}$ for v over $[t_s, t_e]$

- 1 $N \leftarrow N_{root}; \hat{c} = 0;$
- 2 **while** *True* **do**
- 3 $s_1 \leftarrow [c_t(v) || d_t(v)]_{t=t_s, t \notin [t'_s, t'_e]}^{t_e};$
- 4 $s_2 \leftarrow [C_{T_{N_i}} || d_{T_{N_i}}(v)]_{i=0}^{|N_c|-1};$
- 5 $\hat{c} \leftarrow \mathcal{M}_N(\hat{c}, s_1, s_2);$ // Eq. (2)
- 6 **for** $N' \in N.child$ **do**
- 7 **if** $[t_s, t_e] \subseteq T_{N'}$ **then**
- 8 $\hat{N} \leftarrow N';$ **break;**
- 9 **if** no child node contains $[t_s, t_e]$ **then break;**
- 10 **return** $\hat{c};$

Hierarchical Model Index (HM-Index) over recursively nested time intervals, as illustrated in Figure 3. Each node N in the HM-Index has three components: (1) T_N : the time interval it covers; (2) $\mathcal{M}_N$: a lightweight model that predicts vertex coreness for any subinterval of T_N ; (3) $C_N = \{c_{T_N}(v) \mid v \in \mathcal{V}\}$: the set of vertex coreness values over T_N , used by the parent node to improve prediction accuracy. This nested organization lets each child model reuse the parent prediction and refine it with information from a narrower interval. This residual refinement design is formalized in Lemma 4.1.

Time Interval Partition Rules. The HM-Index employs a recursive partitioning scheme: the root node spans the entire temporal range $[1, t_{\max}]$; each parent node divides its interval into p equal subintervals; partitioning terminates when leaf node’s time interval length falls below the given threshold x . For example, in Figure 3, $p = 2$ and $x = \frac{L}{4}$. The HM-Index structure maintains two key properties: (1) *Strict Hierarchy*: for any child node N' of a node N , we have $T_{N'} \subset T_N$. (2) *Disjoint Siblings*: Nodes at the same depth have non-overlapping time intervals.

Model Structure. The regression model $\mathcal{M}_N$ in a node N is a lightweight Multilayer Perceptron (MLP). It consists of four components: (1) A linear transformation layer $\mathbf{W}_0 \in \mathbb{R}^{\dim_h \times 1}$ for single-valued features; (2) A timestamp feature processing layer $\mathbf{W}_1 \in \mathbb{R}^{\dim_h \times 2\lceil \frac{|T_N|}{p} \rceil}$; (3) A time interval feature processing layer $\mathbf{W}_2 \in \mathbb{R}^{\dim_h \times p}$; (4) A feature fusion layer $\mathbf{W}_f \in \mathbb{R}^{1 \times 3 \times \dim_h}$ that merges the three processed features into a unified representation. Here, $\dim_h$ represents the hidden dimension. The model extracts and fuses specialized features from each input type.

Coreness Prediction Algorithm. The coreness prediction algorithm uses a multi-stage approach that traverses the HM-Index from the root, refining predictions at each level. This top-down traversal passes each prediction to the selected child model, which refines it using more localized interval information. We first describe the input to the model $\mathcal{M}_N$, then present the full algorithm that coordinates this hierarchical prediction process.

Given an HM-Index node N , a vertex v , and a query time interval $[t_s, t_e] \subseteq T_N$, the interval may span multiple child nodes of N . Let $N_c = \{N_0, N_1, \dots, N_{|N_c|-1}\}$ be the ordered sequence of these child nodes, covering a combined interval $[t'_s, t'_e]$. Let N_p be the parent of N . The model $\mathcal{M}_N$ predicts the coreness of v as follows.

$$\hat{c}_{[t_s, t_e]}^N(v) = \mathcal{M}_N(\hat{c}, s_1, s_2) = \mathbf{W}_f(\mathbf{W}_0 \hat{c} || \mathbf{W}_1 s_1 || \mathbf{W}_2 s_2) \quad (2)$$

The model $\mathcal{M}_N$ takes three inputs to predict the coreness of a vertex v : $\hat{c}$, s_1 , and s_2 . $\hat{c}$ is the coreness value predicted by the parent node N_p , or 0 if no parent prediction exists. It is used as the current estimate for $\mathcal{M}_N$, while s_1 and s_2 provide interval-specific information for refining that estimate. The vector s_1 encodes the coreness and degree of v at timestamps outside a given interval $[t'_s, t'_e]$, capturing external temporal dynamics. The vector s_2 aggregates v 's coreness and degree across all time intervals in the child nodes N_c , summarizing its structure over time.

Based on the coreness prediction of a single HM-Index node, the complete coreness prediction algorithm based on the HM-Index is outlined in Algorithm 1. It starts by setting the root node N and initializing the predicted coreness $\hat{c}$ to 0 (line 1). Then, Algorithm 1 traverses the HM-Index as follows. For the currently visited HM-Index node N , two feature vectors s_1 and s_2 are computed (lines 3–4), based on which the coreness $\hat{c}$ is updated (line 5). Next, Algorithm 1 checks if there exists a child node N' of N that fully covers the query time interval $[t_s, t_e]$ (lines 6–8). If so, Algorithm 1 visits N' and passes the updated estimate $\hat{c}$ to the child model for further refinement. Otherwise, Algorithm 1 terminates. This root-to-leaf update implements the residual refinement in Lemma 4.1.

Theoretical Error Analysis. We model HM-Index's hierarchical prediction as a sequential refinement process. During the root-to-leaf traversal in Algorithm 1, each node uses increasingly localized temporal features to correct its parent's prediction. This formulation separates the inherited prediction error from the local approximation error introduced at each level. The following lemma gives an error bound for this top-down residual learning under the stated contraction condition. The contraction rate $\beta < 1$ is treated as an assumption of this refinement process and is evaluated empirically in Appendix A. Let $c_{[t_s, t_e]}^*(v)$ denote the exact coreness of v over the query interval $[t_s, t_e]$.

LEMMA 4.1. *Assume each model $\mathcal{M}_{N_j}$ at depth j in Algorithm 1 satisfies the contraction property $\epsilon_j \leq \beta \epsilon_{j-1} + \eta$, where $\beta \in [0, 1)$ is the contraction rate, $\eta \geq 0$ is the irreducible local error, and $\epsilon_j = |\hat{c}^{(j)} - c_{[t_s, t_e]}^*(v)|$. The final prediction error after traversing h levels is strictly bounded by $\epsilon_h \leq \beta^h \epsilon_0 + \frac{\eta(1-\beta^h)}{1-\beta}$.*

PROOF. HM-Index operates as a coarse-to-fine residual chain. At depth j , $\mathcal{M}_{N_j}$ treats the parent prediction $\hat{c}^{(j-1)}$ as a structural prior, utilizing local temporal features to learn a residual correction.

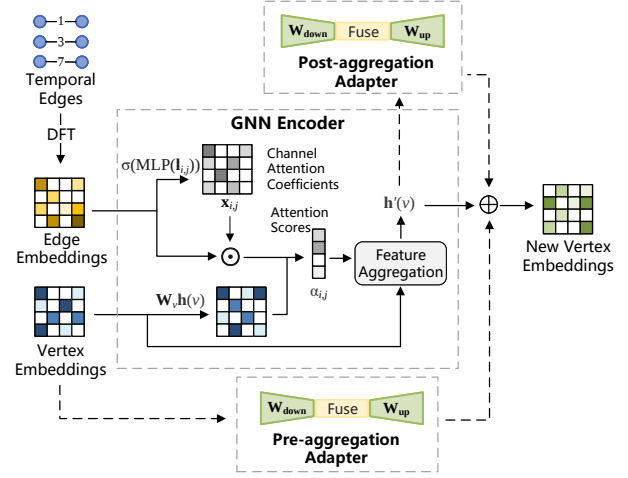


Figure 4: Architecture of AT-GNN

Here, $\beta < 1$ is the contraction assumption, while η bounds the remaining local approximation error.

We prove $\epsilon_j \leq \beta^j \epsilon_0 + \eta \sum_{k=0}^{j-1} \beta^k$ by induction. The base case $j = 0$ is trivial. Assuming this holds for depth j , the contraction property yields $\epsilon_{j+1} \leq \beta \epsilon_j + \eta \leq \beta(\beta^j \epsilon_0 + \eta \sum_{k=0}^{j-1} \beta^k) + \eta = \beta^{j+1} \epsilon_0 + \eta \sum_{k=0}^j \beta^k$, completing the induction.

Evaluating at the final depth h and computing the geometric series yields $\epsilon_h \leq \beta^h \epsilon_0 + \frac{\eta(1-\beta^h)}{1-\beta}$. Thus, the initial propagated error ϵ_0 decays exponentially with depth, while the cumulative local error is bounded by $\frac{\eta}{1-\beta}$. Under the stated contraction condition, this bound prevents error explosion. $\square$

The quantity $\eta/(1-\beta)$ is a conservative upper bound obtained from a uniform local error η . Appendix A discusses the contraction assumption, the risk of overfitting at deeper nodes, and the tightness of this bound, and reports empirical values of β and the per-level approximation errors η_k .

Complexity Analysis. Let $t_{\max}$ be the maximum timestamp of the graph, and x be the maximum length of the time interval for the leaf nodes in the HM-Index. The number of leaf nodes in the HM-Index is bounded by $O(\frac{t_{\max}}{x})$. Since the HM-Index is a balanced tree, the total number of nodes is also bounded by $O(\frac{t_{\max}}{x})$. The height of the HM-Index is $h = \log_p \frac{t_{\max}}{x}$. In the worst case, Algorithm 1 traverses the HM-Index from the root to the leaf node. Hence, the time complexity of Algorithm 1 is bounded by $O(\log \frac{t_{\max}}{x})$. Each node contains a model and a set of coreness values for vertices over the corresponding time interval. Therefore, the space required for a single node is bounded by $O(|\mathcal{M}| + |\mathcal{V}|)$. Consequently, the overall space complexity of HM-Index is bounded by $O(\frac{t_{\max}}{x} \cdot (|\mathcal{M}| + |\mathcal{V}|))$.

4.2 Adaptive Temporal GNN

The Adaptive Temporal Graph Neural Network (AT-GNN) captures structural and temporal graph characteristics while supporting efficient fine-tuning for diverse TICS queries.

Figure 4 shows the architecture of one layer in the AT-GNN. The AT-GNN contains K layers. The model has two main components: the encoder and the adapter. Each layer includes two adapters,

namely the *Pre-aggregation Adapter* and the *Post-aggregation Adapter*. The vertex embedding $h^{(l)}(v)$ in the l -th AT-GNN layer is

$$h^{(l)}(v) = \text{ENC}(h^{(l-1)}(v)) + \text{ADP}_{\text{pre}}(h^{(l-1)}(v)) + \text{ADP}_{\text{post}}(\text{ENC}(h^{(l-1)}(v))) \quad (3)$$

Here, $\text{ENC}(\cdot)$ is the encoder, $\text{ADP}_{\text{pre}}(\cdot)$ and $\text{ADP}_{\text{post}}(\cdot)$ are pre-aggregation and post-aggregation adapters, respectively.

Encoder. For a vertex v , the initial feature $h^{(0)}(v) \in \mathbb{R}^{\text{dim}_v}$ is defined as:

$$h^{(0)}(v) = f_v([q(v)||s_c(v)]), \quad (4)$$

where $q(v)$ indicates whether v is a query vertex (1 if $v \in V_Q$, else 0), and $s_c(v) = [c_t(v)]_{t=t_s}^{t_e}$ is the temporal coreness sequence. The function $f_v(\cdot)$ adjusts the dimension via linear interpolation (for short sequences) or average pooling (for long ones). Vertex features are constructed in the original domain from $q(v)$ and the normalized coreness sequence $s_c(v)$. Temporal interaction patterns are encoded from the time-varying edge-presence sequence using DFT. For an edge (v, v') , the initial embedding $l_{v,v'} \in \mathbb{R}^{\text{dim}_e}$ is

$$l_{v,v'} = [\text{DFT}(f_e([a_t(v, v')]_{t=t_s}^{t_e}))]_{0:\text{dim}_e}. \quad (5)$$

The edge presence sequence $[a_t(v, v')]_{t=t_s}^{t_e}$ indicates whether edge (v, v') exists at time t , where $a_t(v, v') = 1$ if the edge exists and 0 otherwise. Like $f_v(\cdot)$, $f_e(\cdot)$ adjusts the dimension.

LEMMA 4.2. Let $\mathbf{a}, \mathbf{b} \in \{0, 1\}^T$ be two edge presence sequences with Hamming distance $\|\mathbf{a} - \mathbf{b}\|_0 = m$, and let $\phi_d(\mathbf{a}) = [\hat{a}_k]_{k=0}^{d-1}$ denote the truncated DFT amplitude embedding defined in Eq. (5) with $d \leq T$. Then $\|\phi_d(\mathbf{a}) - \phi_d(\mathbf{b})\|_2 \leq \sqrt{m}$.

The $\sqrt{m}$ bound shows that small changes in an edge-presence sequence produce bounded changes in its DFT amplitude embedding. This provides a stability property for the temporal edge representation before it is used by channel attention and neighbor aggregation. After applying DFT, the first dim_e elements of the amplitude vector form the edge embedding. To strengthen temporal modeling, a channel attention mechanism refines the embeddings:

$$\mathbf{x}_{v,v'} = \sigma(\text{MLP}(l_{v,v'})), \quad l'_{v,v'} = l_{v,v'} \odot \mathbf{x}_{v,v'}. \quad (6)$$

The coefficient vector $\mathbf{x}_{v,v'}$ is computed from the DFT amplitude embedding $l_{v,v'}$ of the current edge and serves as an input-dependent channel gate. Through element-wise multiplication, it rescales the original frequency-channel amplitudes while preserving their coordinates before neighbor aggregation.

For vertex aggregation, the encoder calculates attention scores $b_{v,v'}$ by combining vertex and edge features:

$$b_{v,v'} = \text{LeakyReLU}(\mathbf{a}^T \cdot [\mathbf{W}_v \mathbf{h}^{(l-1)}(v) || \mathbf{W}_v \mathbf{h}^{(l-1)}(v') || l'_{v,v'}]). \quad (7)$$

Here, $\mathbf{a}$ is a learnable vector and $\mathbf{W}_v$ projects vertex features. Dynamic edge features $l'_{v,v'}$ capture temporal interactions. The normalized attention scores are then used to aggregate neighboring features to update $h^{(l)}(v)$, as in Eq. (8).

$$\alpha_{v,v'} = \text{softmax}_{v'}(b_{v,v'}) = \frac{\exp(b_{v,v'})}{\sum_{v'' \in \mathcal{N}(v) \cup \{v\}} \exp(b_{v,v''})}, \quad (8)$$

$$\text{ENC}(h^{(l-1)}(v)) = \sum_{v' \in \mathcal{N}(v) \cup \{v\}} \alpha_{v,v'} \cdot h^{(l-1)}(v')$$

Algorithm 2: Forward Propagation of AT-GNN

Input: A temporal graph $\mathcal{G}$, a set of query vertices V_Q , a query time interval T_Q
Output: The embeddings of $\mathcal{V}$

```

1  $\{h^{(0)}(v) \leftarrow f_v([q(v)||s_c(v)]) \mid v \in \mathcal{V}\}$  // Eq. (4)
2  $\{l_{v,v'} \leftarrow [\text{DFT}(f_e([a_t(v, v')]_{t=t_s}^{t_e}))]_{0:\text{dim}_e} \mid e = (v, v'), t \in \mathcal{E}, t \in T_Q\}$  // Eq. (5)
3 for  $l = 1$  to  $K$  do
4   for  $v \in \mathcal{V}$  do
5      $h_{\text{pre}}(v) \leftarrow \text{ADP}(h^{(l-1)}(v))$ ; // Pre-aggregation adapter
6     // encoder Eq. (6) to (8)
7     for  $v' \in \{v\} \cup N_{T_Q}(v)$  do
8        $\mathbf{x}_{v,v'} \leftarrow \sigma(\text{MLP}(l_{v,v'})), l'_{v,v'} \leftarrow l_{v,v'} \odot \mathbf{x}_{v,v'}$ ;
9        $b_{v,v'} = \text{LeakyReLU}(\mathbf{a}^T \cdot [\mathbf{W}_v h^{(l-1)}(v) || \mathbf{W}_v h^{(l-1)}(v') || l'_{v,v'}])$ ;
10       $\alpha_{v,v'} = \text{softmax}_{v'}(b_{v,v'})$ ;
11       $h'(v) \leftarrow \sum_{v' \in \mathcal{N}(v) \cup \{v\}} \alpha_{v,v'} \cdot h^{(l-1)}(v')$ ;
12       $h_{\text{post}}(v) \leftarrow \text{ADP}(h'(v))$ ; // Post-aggregation adapter
13       $h^{(l)} \leftarrow h_{\text{pre}}(v) + h'(v) + h_{\text{post}}(v)$ ;
14 return  $\{h^{(l)}(v) \mid v \in \mathcal{V}\}$ ;

Procedure  $\text{ADP}(h_{\text{in}}(v))$ 
15  $s_Q \leftarrow [\bar{c}(V_Q) || \bar{c}(N(V_Q))]_{\frac{t_s}{t_{\max}}}^{\frac{t_e}{t_{\max}}}$ ;
16  $\tilde{h}(v) \leftarrow [\mathbf{W}_{\text{down}_1} h_{\text{in}}(v) || \mathbf{W}_{\text{down}_2} s_Q]$ ; // Eq. (9)
17  $\mathbf{q} \leftarrow \mathbf{W}_q \tilde{h}(v), \mathbf{k} \leftarrow \mathbf{W}_k \tilde{h}(v), \mathbf{v} \leftarrow \mathbf{W}_v \tilde{h}(v)$ ; // Eq. (9)
18  $\alpha \leftarrow \text{softmax}(\frac{\mathbf{q}^T \mathbf{k}}{\sqrt{\text{dim}_{\text{head}}}})$ ; // Eq. (10)
19 return  $\text{ReLU}(\mathbf{W}_{\text{up}}(\alpha \cdot \mathbf{v}))$ ; // Eq. (10)
```

Adapter. COCO is designed to handle TICS queries under varying graph structures. Fully retraining or fine-tuning the encoder for different queries would be prohibitively expensive. To address this issue, we design a query context-aware adapter. This lightweight trainable module is inserted into each encoder layer, both before and after the aggregation step. This dual-adapter architecture improves generalization by enabling controlled feature adaptation while preserving the original graph representations.

Each adapter includes three parts: dimension reduction, feature fusion, and dimension expansion. In the dimension reduction stage, two inputs are processed. First, vertex embeddings $h_{\text{in}}(v)$ are linearly projected to a lower dimension using $\mathbf{W}_{\text{down}_1}$. Second, query context features are extracted: average coreness of query vertices and their neighbors ($\bar{c}(V_Q)$ and $\bar{c}(N(V_Q))$), and normalized time features $\frac{t_s}{t_{\max}}$ and $\frac{|T_Q|}{t_{\max}}$. These features are combined into a vector s_Q , then projected via $\mathbf{W}_{\text{down}_2}$. After concatenation, three learnable matrices $\mathbf{W}_q$, $\mathbf{W}_k$, and $\mathbf{W}_v$ project the input into query, key, and value vectors, as shown in Eq. (9).

$$\tilde{h}(v) = [\mathbf{W}_{\text{down}_1} h_{\text{in}}(v) || \mathbf{W}_{\text{down}_2} s_Q], \quad (9)$$

$$\mathbf{q} = \mathbf{W}_q \tilde{h}(v), \quad \mathbf{k} = \mathbf{W}_k \tilde{h}(v), \quad \mathbf{v} = \mathbf{W}_v \tilde{h}(v)$$

In the final dimension expansion stage, the output is further transformed by a learnable matrix $\mathbf{W}_{\text{up}}$. The complete adapter output for input $h_{\text{in}}(v)$ is given in Eq. (10):

$$\alpha = \text{softmax}\left(\frac{\mathbf{q}^T \mathbf{k}}{\sqrt{\text{dim}_{\text{head}}}}\right), \quad \text{ADP}(h_{\text{in}}(v)) = \text{ReLU}(\mathbf{W}_{\text{up}}(\alpha \cdot \mathbf{v})) \quad (10)$$

This reduce-then-expand design limits the dimension of the adapter update while retaining query-specific adjustments.

LEMMA 4.3. *For a fixed query context s_Q , a fixed active region of the ReLU, and a linear/identity configuration of the fusion module, the adapter update has jacobian rank at most $\xi = \min\{\dim_{\text{ADP}}, \dim_{\text{head}}\}$. Within this region, it can exactly realize any affine correction $\Delta_Q(\mathbf{h}) = \mathbf{A}_Q \mathbf{h} + \mathbf{b}_Q$ satisfying $\text{rank}([\mathbf{A}_Q \ \mathbf{b}_Q]) \leq \xi$, provided that its output remains componentwise nonnegative.*

Here, $\dim_{\text{ADP}}$ denotes the hidden dimension of the adapter’s bottleneck, and $\dim_{\text{head}}$ is the dimension of the attention head. Lemma 4.3 first bounds the rank of the update produced through the down-projection and up-projection. Within a fixed activation region, its exact-realization result characterizes the query-specific affine corrections represented by the adapter. The bottleneck dimension therefore controls the capacity of query-specific adaptation while keeping the pre- and post-aggregation adapters compact.

Forward Propagation. As shown in Algorithm 2, forward propagation of AT-GNN begins by computing initial embeddings for vertices and edges (lines 1–2). Then, the model performs K layers of message passing. Each layer includes a pre-aggregation adapter, an encoder, and a post-aggregation adapter. In the pre-aggregation adapter, the input embedding $\mathbf{h}^{(l-1)}(v)$ is transformed into an intermediate representation $\mathbf{h}_{\text{pre}}(v)$ using the ADP procedure (line 5). ADP improves embeddings by adding query-specific context. It builds a context vector s_Q from the query vertex set V_Q and its neighbors, applies self-attention, and outputs a refined representation (lines 14–18). Next, the encoder aggregates neighbor information. For each neighbor v' of v , attention scores $\alpha_{v,v'}$ are calculated using channel attention, and used to combine neighbor embeddings (lines 6–10). The result, $\mathbf{h}'(v)$, is further processed by the post-aggregation adapter via ADP (line 11). Finally, the output combines features from all three components (line 12).

5 COCO WORKFLOW

In this section, we present the workflow of COCO, which includes pre-training, fine-tuning, and searching phases. Algorithm 3 constructs HM-Index according to the interval-partition rule in Section 4.1, while the remaining procedures train the components and use them for query processing.

5.1 Pre-training Phase

In the pre-training phase, COCO builds the HM-Index and trains the AT-GNN. We describe their training below.

Construction of the HM-Index. The HM-Index construction algorithm builds a hierarchical tree structure to predict vertex coreness in temporal graphs. Using a top-down strategy, the algorithm recursively divides time intervals into smaller segments. For each resulting time interval, it creates a corresponding tree node and trains a specialized prediction model.

As outlined in Algorithm 3, the procedure starts by initializing a queue L containing the root node. The root node covers the entire time interval $\mathcal{T}$ (line 1). For each node N from the queue, the algorithm first creates a model (line 4) and then randomly samples batches of vertices (line 5). For each vertex v in the batch, it selects a random subinterval of T_N (line 9). The coreness of v over this

Algorithm 3: HM-Index Construction

Input: A temporal graph $\mathcal{G}$, a partition coefficient p , a length threshold x , a learning rate φ , a number of epochs λ , a batch size n_b

Output: Root node of HM-Index

- 1 Initialize queue L with a node N_{root} with entire time interval $\mathcal{T}$;
- 2 **while** $L \neq \emptyset$ **do**
- 3 $N \leftarrow L.\text{dequeue}()$; initialize a model $\mathcal{M}_N$ for N ;
- 4 Sample batches $\{V_1, V_2, \dots, V_n\}$ where each $|V_{b_i}| = n_b$;
- 5 **for** epoch = 1 to λ **do**
- 6 **for** each batch $V_i \in \{V_1, V_2, \dots, V_n\}$ **do**
- 7 $\mathcal{L}_c \leftarrow 0$;
- 8 **for** $v \in V_i$ **do**
- 9 $[t_s, t_e] \leftarrow$ randomly sample subinterval from T_N ;
- 10 $c(v)_{[t_s, t_e]} \leftarrow$ compute the coreness of v ;
- 11 $\hat{c}_{[t_s, t_e]}(v) \leftarrow$ predict using Algorithm 1;
- 12 $\mathcal{L}_c \leftarrow \mathcal{L}_c + \frac{1}{n_b} (\hat{c}(v) - c(v))^2$;
- 13 $\theta(\mathcal{M}_N) \leftarrow \theta(\mathcal{M}_N) - \varphi \cdot \nabla_{\theta} \mathcal{L}_c$;
- 14 **if** $|T_N| > x$ **then**
- 15 Create p child nodes with equal-length subintervals;
- 16 Enqueue child nodes into L ;
- 17 **return** N_{root} ;

subinterval is computed using the core decomposition method [13] and serves as the ground-truth label (line 10). Meanwhile, Algorithm 1 generates the predicted coreness of v (line 11). The training objective is the mean squared error between predictions and labels.

$$\mathcal{L}_c = \frac{1}{n_b} \sum_{i=1}^{n_b} (\hat{c}(v_i) - c(v_i))^2 \quad (11)$$

This loss is calculated for each batch (line 12), and the parameters of the model $\mathcal{M}_N$ are updated accordingly (line 13). If the current node’s time interval has a length greater than or equal to x , it is divided into p subintervals (line 15), and child nodes corresponding to these new intervals are enqueued (line 16). The HM-Index is complete once all eligible nodes are processed.

Training for AT-GNN. Most learning-based community search methods depend on ground-truth communities. However, this approach does not work for temporal graphs because there are no ground-truth communities available for existing temporal graphs [20]. To address this, we propose a contrastive learning method to train AT-GNN in an unsupervised manner. We design two loss functions for training AT-GNN: the cohesive loss and the temporal link loss.

The cohesive loss function is designed to bring the embeddings of potentially related vertices (likely belonging to the same community) closer together in the latent space, while simultaneously pushing unrelated vertices farther apart. To select positive samples, we define the cohesiveness score $\text{CoS}(v)$ of candidate vertex v over T_Q :

$$\text{CoS}(v) = c(v) \cdot \frac{|\{e = (v, u, t) | u \in N(v), t \in T_Q\}|}{|N_{T_Q}(v)|} \quad (12)$$

Here, $c(v)$ is the coreness of v , while the second factor is its average number of temporal interactions per active neighbor in T_Q . Their product measures local structural and temporal cohesiveness.

Sampling Rule 1. To enable contrastive learning in AT-GNN, positive and negative samples are generated for the cohesive loss.

(1) **Positive Samples V_{pos} :** Starting from V_Q , V_{pos} is expanded by repeatedly adding the vertex v' with the highest $\text{CoS}(v')$ from

$N_{T_Q}(V_{\text{pos}}) \setminus V_{\text{pos}}$ until the number of positive samples reaches $|V_{\text{smp}}|$. This frontier expansion keeps V_{pos} connected to V_Q .

(2) **Negative Samples V_{neg} :** V_{neg} is formed by randomly selecting $|V_{\text{smp}}|$ vertices from $\{\mathcal{V} \setminus V_{\text{pos}}\}$.

Once the positive and negative samples are identified, we model the cohesive loss using the margin triplet loss [31]. It minimizes the distance between query vertices and positive samples while enforcing a margin between the query and negative samples.

$$\mathcal{L}_{\text{Coh}} = \frac{1}{|V_{\text{pos}}| \cdot |V_{\text{neg}}|} \cdot \sum_{u \in V_{\text{pos}}} \sum_{v \in V_{\text{neg}}} \max(0, \text{dis}(u, V_Q) - \text{dis}(v, V_Q) + \epsilon) \quad (13)$$

Here, $\text{dis}(\cdot)$ computes the average L2 distance between a vertex and query vertices. For example, $\text{dis}(v, V_Q) = \frac{1}{|V_Q|} \cdot \sum_{v' \in V_Q} \|\mathbf{h}(v) - \mathbf{h}(v')\|$. ϵ is the margin value.

A vertex may have a different number of connections with its neighbors. The idea behind the temporal link loss is to enhance the similarity of vertices with frequent connections while distinguishing weakly linked vertices. To measure the strength of the connection between two vertices, we define a function of link strength.

$$E(v, u) = \{e = (v, u, t) | e \in \mathcal{E}, t \in T_Q\},$$

$$\text{LinkS}(v, u) = \frac{|E(v, u)|}{\max_{(v, u, t) \in E(v, u)} t - \min_{(v, u, t) \in E(v, u)} t + 1} \quad (14)$$

Sampling Rule 2. The samples for the temporal link loss are triples, denoted as V_{temp} . Each triple $(v, v_p, v_n) \in V_{\text{temp}}$ consists of an anchor vertex, a positive vertex, and a negative vertex.

(1) **Anchor Vertex v :** v is randomly selected from $\mathcal{V}$.

(2) **Positive Vertex v_p :** Among the neighbors of v , the positive vertex is $v_p = \arg \max_{v' \in N_{T_Q}(v)} \text{LinkS}(v, v')$.

(3) **Negative Vertex v_n :** Among the neighbors of v , the negative vertex is $v_n = \arg \min_{v' \in N_{T_Q}(v)} \text{LinkS}(v, v')$.

Based on the sampled triples, the temporal link loss is defined in Eq.(15). Finally, we combine the cohesive loss and the temporal link loss as shown in Eq.(16).

$$\mathcal{L}_{\text{Temp}} = \frac{1}{|V_{\text{temp}}|} \cdot \sum_{(v_a, v_p, v_n) \in V_{\text{temp}}} \max(0, \text{dis}(v_a, v_p) - \text{dis}(v_a, v_n) + \epsilon) \quad (15)$$

$$\mathcal{L} = \mathcal{L}_{\text{Coh}} + \delta \mathcal{L}_{\text{Temp}} \quad (16)$$

The overall training process of AT-GNN is presented in Algorithm 4. The algorithm starts by generating several query batches (line 1). Each query has a different query vertex and a different time interval. This allows AT-GNN to learn community information across various time intervals. For each query, the algorithm samples data for the cohesive loss and computes the loss (lines 7-8). It then samples data for the temporal link loss and computes the corresponding loss (lines 9-10). The losses are combined (line 11), and AT-GNN parameters are updated after each batch (line 12).

5.2 Fine-tuning Phase

In the fine-tuning phase, the adapter module in AT-GNN plays a key role. When a query arrives, COCO avoids heavy query overhead by fine-tuning only the parameters in the adapter module. The process of fine-tuning is shown in Algorithm 5.

Algorithm 4: AT-GNN Pre-training

Input: A temporal graph $\mathcal{G}$, a learning rate φ , a number of epochs λ , a batch size n_b , an initialized AT-GNN model $\mathcal{M}_{\text{AT}}$

Output: The trained model $\mathcal{M}_{\text{AT}}$

```

1 Generate query batches  $\{Q_1, Q_2, \dots, Q_n\}$  where  $Q_i$  has  $n_b$  queries;
2 for epoch = 1 to  $\lambda$  do
3   for each batch  $Q_i \in \{Q_1, Q_2, \dots, Q_n\}$  do
4      $\mathcal{L} \leftarrow 0$ ;
5     for each query  $(V_Q, T_Q) \in Q_i$  do
6        $\{\mathbf{h}^{(l)}(v) \mid v \in \mathcal{V}\} \leftarrow$  Algorithm 2;
7        $V_{\text{pos}}, V_{\text{neg}} \leftarrow$  sample by rule 1;
8        $\mathcal{L}_{\text{Coh}} \leftarrow \frac{1}{n_b} \cdot \frac{1}{|V_{\text{pos}}| \cdot |V_{\text{neg}}|} \sum_{u \in V_{\text{pos}}} \sum_{v \in V_{\text{neg}}} \max(0, \text{dis}(u, V_Q) - \text{dis}(v, V_Q) + \epsilon)$ ;
9        $V_{\text{temp}} \leftarrow$  sample by rule 2;
10       $\mathcal{L}_{\text{Temp}} \leftarrow \frac{1}{n_b} \cdot \frac{1}{|V_{\text{temp}}|} \sum_{(v_a, v_p, v_n) \in V_{\text{temp}}} \max(0, \text{dis}(v_a, v_p) - \text{dis}(v_a, v_n) + \epsilon)$ ;
11       $\mathcal{L} \leftarrow \mathcal{L}_{\text{Coh}} + \delta \mathcal{L}_{\text{Temp}}$ ;
12       $\theta(\mathcal{M}_{\text{AT}}) \leftarrow \theta(\mathcal{M}_{\text{AT}}) - \varphi \cdot \nabla_{\theta} \mathcal{L}$ ;
13 return  $\mathcal{M}_{\text{AT}}$ ;

```

First, COCO predicts the coreness of all vertices using HM-Index via Algorithm 1 (line 1). Then it samples vertices based on Rule 1 and Rule 2 from Section 5.1 (lines 2). For each fine-tuning step, COCO generates the vertex embeddings using AT-GNN (line 4). Next, sampled vertices are used to compute the loss (line 5). The key difference between pre-training and fine-tuning lies in parameter updates. During pre-training, all parameters in AT-GNN are updated. However, during fine-tuning, the parameters in the GNN encoder are frozen. Only the parameters in the adapter module are updated based on the computed loss (line 6).

Freezing the GNN encoder avoids redundant parameter updates and reduces fine-tuning costs. Updating only the adapter enables rapid adaptation to new queries while retaining knowledge learned during pre-training, supporting robust and accurate predictions with limited query-specific data.

5.3 Searching Phase

The searching phase identifies the final community for TICS. COCO progressively expands the result set by adding vertices whose embeddings are similar to those of query vertices. Our adaptive search algorithm dynamically adjusts the termination threshold.

We achieve this dynamic adjustment through an adaptive update function (Eq.(17)). Based on small-world theory [15, 25], community members exhibit strong locality, so we first extract an r -hop subgraph around query vertices, denoted as V_r . Initially, the threshold τ is set to the average embedding distance between vertices in the r -hop subgraph $G_r = (V_r, E_r)$ and query vertices V_Q : $\tau = \frac{\sum_{v \in V_r} \text{dis}(v, V_Q)}{|V_r - V_Q|}$. During each iteration, COCO selects the vertex with the minimum distance and adds it to the result set C . For the n -th iteration, when a new vertex is added to C , the threshold τ is updated according to the following function:

$$\tau_n = \rho_n \cdot \tau_{n-1} + (1 - \rho_n) \cdot \frac{\sum_{v \in C} \text{dis}(v, V_Q)}{|C|}, \quad \rho_n = \cos\left(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}\right) \quad (17)$$

Algorithm 5: COCO Fine Tuning & Searching

Input: A temporal graph $\mathcal{G}$, a set of query vertices V_Q , a query time interval T_Q , a learning rate φ , a number of fine tuning λ , the AT-GNN $\mathcal{M}_{AT}$ model, adapter modules $\mathcal{M}_{ADP}$, HM-Index root node N_{root}

Output: The resulting community C

// Fine-tuning Phase

- 1 $\{\hat{c}(v) | v \in \mathcal{V}\} \leftarrow$ predict corenesses using N_{root} via Algorithm 1;
- 2 $V_{pos}, V_{neg}, V_{temp} \leftarrow$ sample by rules 1 and 2;
- 3 **for** $epoch = 1$ **to** λ **do**
- 4 $\mathcal{L} \leftarrow 0$; $\{\mathbf{h}^{(l)}(v) | v \in \mathcal{V}\} \leftarrow$ infer with $\mathcal{M}_{AT}$ via Algorithm 2;
- 5 $\mathcal{L} \leftarrow$ compute the loss function based on $V_{pos}, V_{neg}, V_{temp}$;
- 6 $\theta(\mathcal{M}_{ADP}) \leftarrow \theta(\mathcal{M}_{ADP}) - \varphi \cdot \nabla_{\theta} \mathcal{L}$;

// Searching Phase

- 7 $C \leftarrow V_Q$; $\{\mathbf{h}^{(l)}(v) | v \in \mathcal{V}\} \leftarrow$ infer with $\mathcal{M}_{AT}$ via Algorithm 2;
- 8 $V_r \leftarrow r$ -hop neighbors of V_Q ;
- 9 $\tau \leftarrow \frac{\sum_{v \in V_r} dis(v, V_Q)}{|V_r - V_Q|}$; $min_dis \leftarrow 0$;
- 10 **while** $min_dis < \tau$ **do**
- 11 $C \leftarrow C \cup \arg \min_{v \in N_{T_Q}(C) \cap V_r} dis(v, V_Q)$;
- 12 $min_dis \leftarrow \min_{v \in N_{T_Q}(C) \cap V_r} dis(v, V_Q)$;
- 13 $\tau \leftarrow \cos(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}) \cdot \tau + (1 - \cos(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|})) \cdot \frac{\sum_{v' \in C} dis(v', V_Q)}{|C|}$;
- 14 **return** C ;

Here, ρ_n smooths the update by combining the previous threshold with the current average distance in C . This prevents abrupt changes as C grows and allows the threshold to adapt to the current community instead of using a fixed cutoff. The search terminates when the minimum distance among the remaining candidates exceeds τ_n . The following lemma characterizes the smoothing factor as $|C|$ grows and the limiting threshold of this stopping rule.

LEMMA 5.1. *The smoothing factor $\rho_n = \cos(\frac{\pi}{2} \cdot \frac{|C|}{|V_r|})$ decreases monotonically with community size $|C|$ and satisfies $\lim_{|C| \rightarrow |V_r|} \rho_n = 0$, guaranteeing that τ converges to $\frac{\sum_{v \in C} dis(v, V_Q)}{|C|}$.*

PROOF. As $|C|$ increases from $|V_Q|$ to $|V_r|$, $\frac{\pi}{2} \cdot \frac{|C|}{|V_r|}$ grows from $\frac{\pi}{2} \cdot \frac{|V_Q|}{|V_r|}$ to $\frac{\pi}{2}$. Within this interval, the cosine function is strictly decreasing, which means ρ_n strictly decreases and approaches 0. In the update equation, when ρ_n approaches 0, the term $\rho_n \cdot \tau_{n-1}$ becomes negligible while $(1 - \rho_n)$ tends toward 1. Consequently, τ_n becomes increasingly determined by the current average distance $\frac{\sum_{v \in C} dis(v, V_Q)}{|C|}$. Since vertices are added in order of increasing distance, this average distance is non-decreasing and eventually stabilizes, causing τ to converge to this stabilized average distance. $\square$

The cosine form is chosen for its slope. Early in the search, it changes slowly and keeps the threshold stable for the small seed set. Later, it decreases faster and gives more weight to the distances observed in C , which curbs excessive expansion.

Overall, the search process is presented in lines 7-14 of Algorithm 5. First, the result set is initialized with query vertices. Embeddings of all vertices are obtained through the inference of AT-GNN (line 7). Next, the r -hop subgraph of is extracted (line 8). The search threshold is then set to the average distance within the subgraph (line 9). The algorithm adds the neighboring vertex with the smallest distance into the result set (line 11). The minimum distance and threshold are updated (lines 12 and 13). Search stops when the minimum distance exceeds the threshold.

Table 2: Statistics of networks

Dataset	Abbr.	$ \mathcal{V} $	$ \mathcal{E} $	t_{max}
CollegeMsg	CM	1.8K	59.8K	193
MathOverflow	MO	24.8K	506.5K	2351
Superuser	SU	194.1K	1.4M	2774
WikiTalk	WT	1.1M	7.8M	2321
Youtube	YT	3.2M	9.3M	226
Flickr	FL	2.3M	33.1M	394
StackOverflow	SO	2.6M	63.5M	2774
Wikipedia	WP	6.9M	129.9M	298

5.4 Time Complexity Analysis

Pre-training Phase The time complexity contains two components. First, the HM-Index tree construction involves building a tree structure with depth $h = \log_p \frac{t_{max}}{x}$, where t_{max} is the maximal timestamp, x is the leaf interval length, and p is the branching factor. The core decomposition process requires $O(|\mathcal{V}| + |\mathcal{E}|)$ time in total across all timestamps. The HM-Index structure contains $O(\frac{t_{max}}{x})$ nodes. For each node, the training process requires $O(|\mathcal{V}| \cdot \log \frac{t_{max}}{x})$ time per epoch. This leads to an overall time complexity of $O(|\mathcal{E}| + \lambda \cdot \frac{t_{max}}{x} \cdot |\mathcal{V}| \cdot \log \frac{t_{max}}{x})$. Second, the AT-GNN processes K GNN layers with feature dimension d . Each layer requires $O(|\mathcal{V}| + |\mathcal{E}|)$ time. Training on Q queries, this results in a time complexity of $O(\lambda \cdot Q \cdot K \cdot (|\mathcal{V}| + |\mathcal{E}|))$. Overall, the total time complexity is $O(\lambda \cdot \frac{t_{max}}{x} \cdot |\mathcal{V}| \cdot \log \frac{t_{max}}{x} + \lambda \cdot Q \cdot K \cdot (|\mathcal{V}| + |\mathcal{E}|))$.

Fine-tuning Phase. With the encoder frozen, adapter updates dominate fine-tuning and require $O(\lambda \cdot Q \cdot K \cdot |\mathcal{V}|)$ time.

Searching Phase. HM-Index predicts vertex corenesses in the r -hop subgraph in $O(|V_r| \cdot \log \frac{t_{max}}{x})$ time. AT-GNN generates vertex embeddings, which are then sorted, taking $O(K \cdot (|V_r| + |E_r|) + |V_r| \cdot \log |V_r|)$ time. Search takes $O(|V_r|)$ iterations, giving an overall time complexity of $O(K \cdot (|V_r| + |E_r|) + |V_r| \cdot (\log |V_r| + \log \frac{t_{max}}{x}))$.

6 PERFORMANCE EVALUATION

This section evaluates COCO. We use eight real-world temporal networks summarized in Table 2. Experiments run on an Ubuntu server with an Intel i9-10900K CPU, 128GB memory, and an Nvidia RTX 3090 GPU. Additional dataset information and experimental results are provided in the *technical report* [23].

Competitors. We compare COCO with seven competitors.

- *QTCS* [20]. A non-learning method that uses time-constrained personalized pagerank to find a β -temporal proximity core containing the query vertex.
- *Temporal k-core* [43]. A non-learning method that identifies a k -core in the projected graph of the given time interval. We employ an indexing-based method [44] to speed up its search process.
- *TransZero* [35]. An unsupervised learning framework for community search using graph transformers.
- *CSFormer* [39]. An unsupervised learning-based approach for static graphs that adds vertices whose predicted corenesses are higher than those of the query vertices into the community.
- *IACS* [4]. An inductive learning-based method for community search across different static graphs.
- *CS-TGN* [9]. A supervised learning-based method designed for community search over temporal graphs.

Table 3: Community quality comparison for different methods

Datasets	CM			MO			SU			WT			YT			FL			SO			WP			Avg +/-		
Metrics	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM
COCO	0.0064	0.5625	0.0375	0.0018	0.5715	0.0280	0.0011	0.5872	0.0114	0.0042	0.6441	0.0085	0.0065	0.6862	0.0045	0.0049	0.7624	0.0020	0.0097	0.5225	0.0042	0.0123	0.5947	0.0011	-	-	-
Temporal k -core	0.0044	0.6771	0.0219	0.0006	0.7292	0.0090	0.0001	0.6762	0.0010	0.0016	0.8304	0.0028	0.0008	0.7443	5.1x10 ⁻⁴	0.0001	0.8263	6.3x10 ⁻⁵	0.0031	0.7231	0.0012	0.0051	0.8308	4.2x10 ⁻⁴	-69.8%	+23.4%	-73.4%
QTCS	0.0048	0.6332	0.0255	0.0011	0.6572	0.0161	0.0006	0.6580	0.0058	0.0022	0.7408	0.0041	0.0028	0.8676	0.0016	0.0033	0.8695	0.0011	0.0045	0.7786	0.0017	OOM	OOM	OOM	-43.2%	+20.6%	-48.6%
TransZero	0.0026	0.8433	0.0051	0.0002	0.8999	0.0021	9.32x10 ⁻⁵	0.9293	6.9x10 ⁻⁴	0.0001	0.8838	1.8x10 ⁻⁴	0.0004	0.9650	1.9x10 ⁻⁴	0.0003	0.9865	4.5x10 ⁻⁵	7.02x10 ⁻⁵	0.9780	2.4x10 ⁻⁵	OOM	OOM	OOM	-89.1%	+51.4%	-94.8%
CS-Former	0.0007	0.9037	-0.0085	0.0003	0.9490	4.9x10 ⁻⁴	8.05x10 ⁻⁶	0.9660	-5.8x10 ⁻⁴	0.0002	0.9388	2.1x10 ⁻⁴	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-94.7%	+59.2%	-105.9%
IACS	0.0030	0.9500	-0.0036	0.0002	0.8340	0.0019	0.0001	0.9895	-6.8x10 ⁻⁴	0.0004	0.6850	4.5x10 ⁻⁴	1.98x10 ⁻⁷	0.8785	7.7x10 ⁻⁷	8.04x10 ⁻⁸	0.9872	2.2x10 ⁻⁷	1.03x10 ⁻⁶	0.8942	9.3x10 ⁻⁷	OOM	OOM	OOM	-90.7%	+45.5%	-100.5%
CS-TGN	0.0024	0.9059	0.0014	5.12x10 ⁻⁵	0.7008	0.0017	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-78.6%	+41.8%	-95.0%
UTCS	0.0019	0.6876	0.0118	0.0016	0.6678	0.0209	0.0003	0.6481	0.0030	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-51.9%	+16.5%	-56.0%

* A community is considered to be of higher quality when it exhibits a **higher TD**, a **lower TC**, and a **higher TM**.

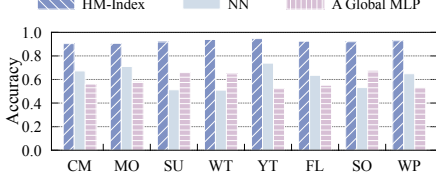


Figure 5: Coreness prediction accuracy

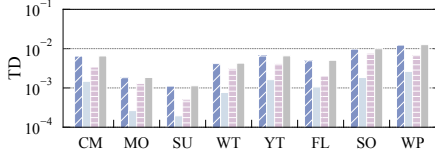


Figure 6: Effect of coreness prediction on TD

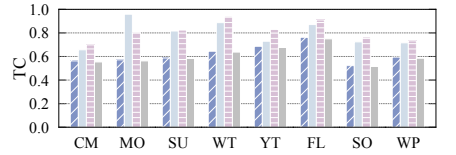


Figure 7: Effect of coreness prediction on TC

- **UTCS** [45]. An unsupervised method for temporal graphs that pre-trains on temporal dynamics and subgraph knowledge to identify communities using proximity scores.

Metrics. Given the absence of ground-truth datasets for temporal graphs, we follow QTCS [20] and use Temporal Density (TD) and Temporal Conductance (TC). We additionally use Temporal Modularity (TM) for community search [14], which extends classic modularity [27] by treating interaction counts in the query interval as edge weights. For a query interval $[t_s, t_e]$, let $\mathcal{E}_Q = \mathcal{E}_{[t_s, t_e]}$. For a retrieved community C , these metrics are

$$TD(C) = \frac{2 \cdot |\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} \mid v, u \in C\}|}{|C| \cdot (|C| - 1) \cdot (t_e - t_s + 1)} \quad (18)$$

$$TC(C) = \frac{|\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]} \mid v \in C, u \notin C\}|}{\min \left\{ \sum_{v \in C} |\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]}\}|, \sum_{v \notin C} |\{(v, u, t) \in \mathcal{E}_{[t_s, t_e]}\}| \right\}} \quad (19)$$

$$TM(C) = \frac{|\{(v, u, t) \in \mathcal{E}_Q \mid v, u \in C\}|}{|\mathcal{E}_Q|} - \left(\frac{\sum_{v \in C} |\{(v, u, t) \in \mathcal{E}_Q\}|}{2|\mathcal{E}_Q|} \right)^2 \quad (20)$$

TD measures the internal cohesion of C as the fraction of possible internal temporal edges that occur during the query interval. **TC** measures the external separation of C by comparing the number of temporal edges crossing its boundary with the smaller incident temporal-edge volume. **TM** compares the fraction of interactions within C with its expectation under a degree-preserving null model. High-quality communities thus have high TD and TM and low TC.

Settings. We set the HM-Index partition coefficient $p = 4$, leaf threshold $x = 40$, and learning rate to 0.001. AT-GNN samples 200 vertices and 200 time intervals randomly for training. COCO runs for 150 epochs with early stopping and uses 10 fine-tuning steps. Search uses a 5-hop subgraph per query vertex and $\gamma = 1$ in Eq.(17). Baselines follow published defaults: QTCS uses standard settings, temporal k -core sets k to the query vertex coreness, and label-dependent IACS [4] and CS-TGN [9] use TransZero [35] pseudo-labels. Exp-1 uses intervals covering 10%–20% of the graph lifespan. Each experiment averages 100 random single-vertex queries.

6.1 Effectiveness Evaluation

First, we evaluate the overall effectiveness of COCO.

Exp-1: Community Quality Comparison. Table 3 compares quality across datasets. COCO leads all baselines in cohesion (TD), separation (TC), and modularity (TM). Against QTCS, its improvements are 43.22%, 20.59%, and 48.58%, respectively; against TransZero, they are 89.14%, 51.40%, and 94.78%. HM-Index supplies cohesive structural information, while AT-GNN captures temporal interaction patterns. Temporal k -core and QTCS impose structural constraints that can exclude well-connected vertices, whereas TransZero, IACS, and CS-TGN lack explicit structural cohesion modeling.

Exp-2: Coreness Prediction Accuracy of HM-Index. To justify the hierarchical design of HM-Index, we compare it with a global MLP trained on random intervals from the full graph lifespan and a nearest-neighbor (NN) method. NN caches exact coreness for the same hierarchical anchor intervals used by HM-Index ($p = 4$ and $x = 40$). For each T_Q , NN selects the cached anchor with the largest interval IoU and returns the exact coreness stored for that anchor. We measure accuracy by $1 - \text{MAPE}$, where $\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{c(v)_i - \hat{c}(v)_i}{c(v)_i} \right|$. Figure 5 shows that HM-Index achieves over 90% accuracy and outperforms both baselines. Global MLP struggles with topological variation across query intervals. NN is sensitive to temporal k -core changes: even highly overlapping intervals may differ in critical edges whose removal triggers cascading core changes. The hierarchy helps HM-Index capture these variations.

Exp-3: Effect of Coreness Accuracy on Community Quality. We compare downstream TD and TC using coreness from HM-Index, NN, Global MLP, and exact computation. Figures 6 and 7 show that NN and Global MLP yield lower TD and higher TC across all datasets because prediction errors distort AT-GNN features. HM-Index yields community quality nearly identical to exact coreness. These results suggest two conclusions. First, accurate coreness prediction is necessary for identifying cohesive communities. Second, the prediction accuracy of HM-Index is high enough to preserve strong downstream community quality, without incurring the heavy cost of online exact core decomposition. On MO, NN has higher average coreness accuracy than Global MLP but yields worse TD and TC because the two predictors produce different error distributions. Interval mismatch can produce a few large NN errors that strongly distort the structural features used by AT-GNN, whereas Global MLP produces smoother errors that AT-GNN’s

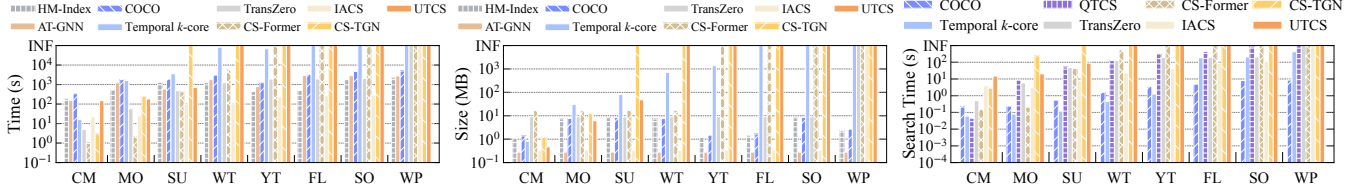
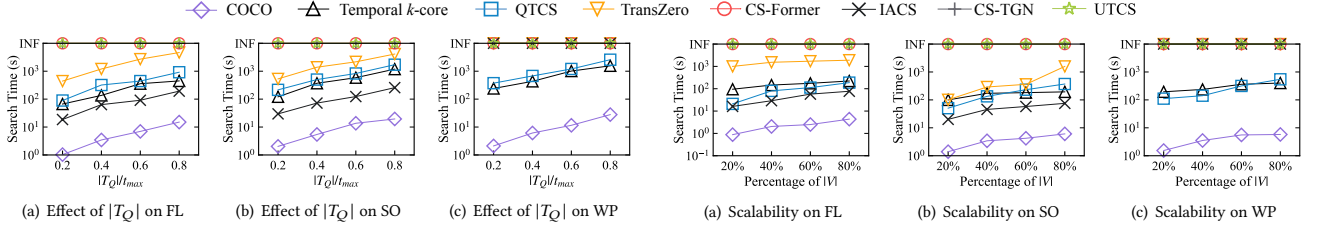


Figure 8: Training/Construction time

Figure 9: Model/Index Size

Figure 10: Search time



(a) Effect of $|T_Q|$ on FL

(b) Effect of $|T_Q|$ on SO

(c) Effect of $|T_Q|$ on WP

Figure 11: Effect of $|T_Q|$

(a) Scalability on FL

(b) Scalability on SO

(c) Scalability on WP

Figure 12: Scalability

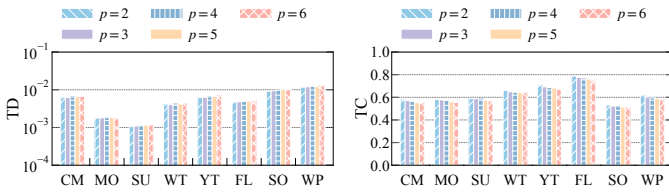


Figure 13: Effect of p on TD

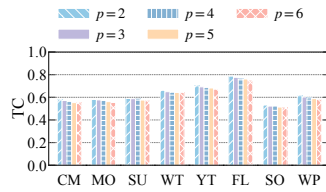


Figure 14: Effect of p on TC

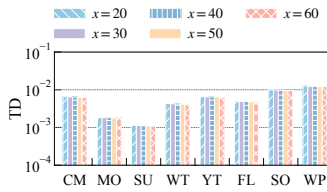


Figure 15: Effect of x on TD

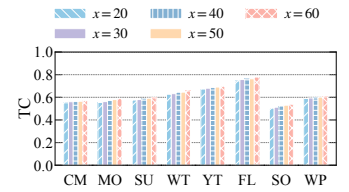


Figure 16: Effect of x on TC

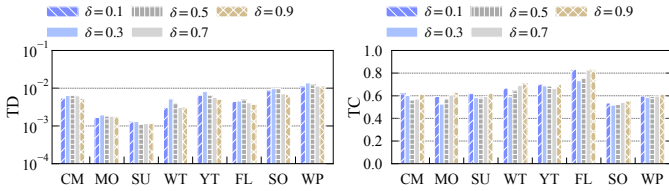


Figure 17: Effect of δ on TD

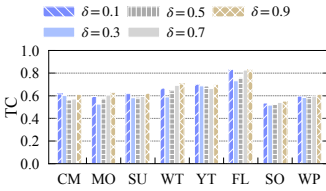


Figure 18: Effect of δ on TC

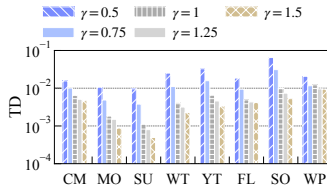


Figure 19: Effect of γ on TD

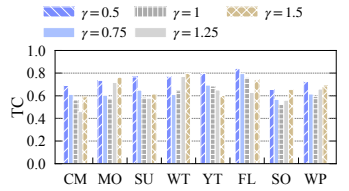


Figure 20: Effect of γ on TC

learned attention can partly tolerate. Therefore, average coreness accuracy alone does not determine downstream community quality.

6.2 Efficiency Evaluation

We compare efficiency across methods, marking a method as INF if it runs out of memory or exceeds 10^5 seconds.

Exp-4: Offline Cost Comparison. Figures 8 and 9 compare offline time and storage. **Training / Construction Time** (Figure 8). We report index construction time for temporal k -core and training time for COCO, TransZero, CS-Former, IACS, and CS-TGN. COCO is comparable to other learning methods. Temporal k -core takes much longer on large datasets because it computes coreness across all intervals. **Model / Index Size** (Figure 9). COCO has a smaller AT-GNN than other learning models. Its lightweight node-level MLPs keep HM-Index storage low despite the tree overhead. Temporal k -core stores every vertex's coreness across all intervals, so its storage grows rapidly with graph size.

Exp-5: Online Search Time Comparison. COCO's search time includes fine-tuning. Figure 10 shows that COCO is approximately two orders of magnitude faster than QTCS. Compared with temporal k -core, COCO is slower on small graphs (CM, MO, SU, WT, and YT) but faster on large graphs (FL, SO, and WP). On small graphs, temporal k -core uses index lookup. On large graphs, indexing costs force temporal k -core to use costly online decomposition.

Exp-6: Effect of $|T_Q|$ on Search Time. We vary $|T_Q|$ from 20% to 80% of $t_{\max}$ on FL, SO, and WP. Figure 11 shows increasing search time for all methods because longer intervals involve more vertices and edges. COCO remains at least one order of magnitude faster than the baselines, benefiting from efficient fine-tuning.

Exp-7: Scalability. Figure 12 evaluates scalability on FL, SO, and WP by randomly sampling vertices to create graphs of different sizes. All methods slow down as larger graphs generally contain larger communities. COCO retains at least a $10\times$ speed advantage over all baselines, demonstrating scalability.

6.3 Parameters Analysis

We vary four parameters to assess sensitivity. HM-Index uses partition coefficient p and leaf threshold x . AT-GNN uses δ to balance losses in Eq. (16) and γ to control search in Eq. (17).

Exp-8: Effect of Partition Coefficient p . Figures 13 and 14 show the effect of p on TD and TC; Figure 23(a) in Appendix D reports TM. Increasing p improves TD, TC, and generally TM across all datasets. More branches yield finer intervals, helping node models capture temporal and structural variations and predict coreness accurately. Quality changes remain limited, indicating robustness to p . We use $p = 4$ to balance prediction reliability and index complexity.

Exp-9: Effect of Leaf Node Threshold x . Figures 15 and 16 show the effect of x on TD and TC; Figure 23(b) in Appendix D reports

Table 4: Component Ablation

Methods	CM			MO			SU			WT			YT			FL			SO			WP			Avg. +/-		
Metrics	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM
COCO (full)	0.0064	0.5625	0.0375	0.0018	0.5715	0.0280	0.0011	0.5872	0.0114	0.0042	0.6441	0.0085	0.0065	0.6862	0.0045	0.0049	0.7624	0.0020	0.0097	0.5225	0.0042	0.0123	0.5947	0.0011	-	-	-
w/o HM-Index (use single MLP)	0.0034	0.6982	0.0298	0.0013	0.8010	0.0161	0.0005	0.8240	0.0106	0.0031	0.9407	0.0060	0.0042	0.8292	0.0035	0.0020	0.9095	0.0012	0.0077	0.7652	0.0032	0.0066	0.7325	0.0007	-39.8%	+32.5%	-28.2%
w/o AT-GNN (use GAT)	0.0051	0.5935	0.0333	0.0015	0.7685	0.0230	0.0010	0.5953	0.0092	0.0038	0.7887	0.0066	0.0057	0.7803	0.0041	0.0046	0.8742	0.0015	0.0093	0.5777	0.0036	0.0097	0.7629	0.0009	-12.2%	+16.2%	-17.0%
w/o Adapter (use linear layer)	0.0057	0.6208	0.0336	0.0016	0.6522	0.0262	0.0011	0.6037	0.0103	0.0038	0.6491	0.0078	0.0065	0.6943	0.0041	0.0048	0.8639	0.0019	0.0084	0.5237	0.0037	0.0110	0.6355	0.0010	-7.5%	+6.2%	-8.7%
w/o DFT (use raw temporal feature)	0.0059	0.5989	0.0351	0.0017	0.6064	0.0254	0.0010	0.6302	0.0108	0.0039	0.7080	0.0079	0.0061	0.7508	0.0042	0.0046	0.8347	0.0019	0.0090	0.5711	0.0039	0.0115	0.6459	0.0010	-6.5%	+8.3%	-7.2%
w/o $\mathcal{L}_{Coh}$ (use $\mathcal{L}_{Temp}$ only)	0.0041	0.5856	0.0337	0.0017	0.5959	0.0237	0.0009	0.7158	0.0098	0.0036	0.7176	0.0074	0.0041	0.7662	0.0039	0.0041	0.8366	0.0017	0.0066	0.6392	0.0035	0.0113	0.5952	0.0009	-20.8%	+10.7%	-14.6%
w/o $\mathcal{L}_{Temp}$ (use $\mathcal{L}_{Coh}$ only)	0.0056	0.6655	0.0302	0.0017	0.6958	0.0221	0.0009	0.6470	0.0095	0.0035	0.7731	0.0073	0.0059	0.6938	0.0042	0.0039	0.9249	0.0017	0.0082	0.5892	0.0034	0.0118	0.7635	0.0010	-12.2%	+16.7%	-14.7%
w/o Coreness Feature (use Node2Vec)	0.0051	0.6101	0.0333	0.0014	0.6565	0.0241	0.0010	0.6401	0.0104	0.0028	0.6955	0.0072	0.0060	0.9055	0.0041	0.0045	0.9482	0.0017	0.0077	0.5794	0.0035	0.0109	0.6431	0.0009	-16.6%	+14.5%	-13.4%

TM. Community quality slightly declines as x increases from 20 to 60 across all datasets. A smaller x yields finer temporal partitions in HM-Index leaf nodes, allowing regression models to capture short-term structural variations in vertex coreness. Larger x produces coarser intervals that may miss these temporal patterns and reduce prediction accuracy. We choose $x = 40$ to balance prediction quality and indexing efficiency.

Exp-10: Effect of δ . We vary δ , which balances the cohesive and temporal link losses in Eq. (16). Figures 17 and 18 report TD and TC; Figure 23(c) in Appendix D reports TM. All three metrics generally improve as δ approaches 0.5 and degrade beyond it. Excess temporal loss weight ($\delta > 0.5$) weakens structural cohesion. Thus, $\delta = 0.5$ balances cohesion and temporal consistency on most datasets.

Exp-11: Effect of γ . We vary γ from 0.5 to 1.5 to study threshold decay in Eq. (17). Figures 19 and 20 report TD and TC, while Figure 23(d) in Appendix D reports TM. Larger γ produces larger communities, generally reducing TD and TM, while TC varies by dataset. COCO remains robust with the default $\gamma = 1$.

6.4 Ablation Study

Exp-12: Component Ablation. Table 4 evaluates components, losses, and input features using TD, TC, and TM. (1) Replacing HM-Index with a single MLP decreases TD by 39.8% and TM by 28.2% on average, supporting hierarchical coreness prediction. (2) Replacing AT-GNN with GAT decreases TD by 12.2%, increases TC by 16.2%, and decreases TM by 17.0%. (3) Replacing the adapter with a linear layer removes query context (s_Q) and the bottleneck, degrading all three metrics. (4) Replacing DFT with raw temporal features decreases TD by 6.5%, increases TC by 8.3%, and decreases TM by 7.2%, supporting frequency-domain interaction modeling. (5) Removing $\mathcal{L}_{Coh}$ and $\mathcal{L}_{Temp}$ decreases TM by 14.6% and 14.7%, respectively. The two losses provide distinct structural and temporal supervision. (6) Replacing coreness with Node2Vec degrades all three metrics, supporting coreness as a signal for cohesive community search. These results confirm each component’s contribution.

6.5 Case Study

Exp-13: Case Study. We conduct a case study on a temporal co-authorship network from DBLP [3], where vertices represent authors and timestamped edges denote publication collaborations. In the case study, we set the query time interval from 2020 to 2025. Figure 21 compares COCO, temporal k -core, QTCS, UTCS, and a frequency-only variant, overlaying temporal k -core and QTCS in Figure 21(b). The frequency-only variant repeatedly adds the frontier author with the highest average interaction frequency. All four panels share one force-directed layout with fixed vertex positions, so differences reflect the returned communities rather than independent layouts. Unselected vertices and edges are gray, the query

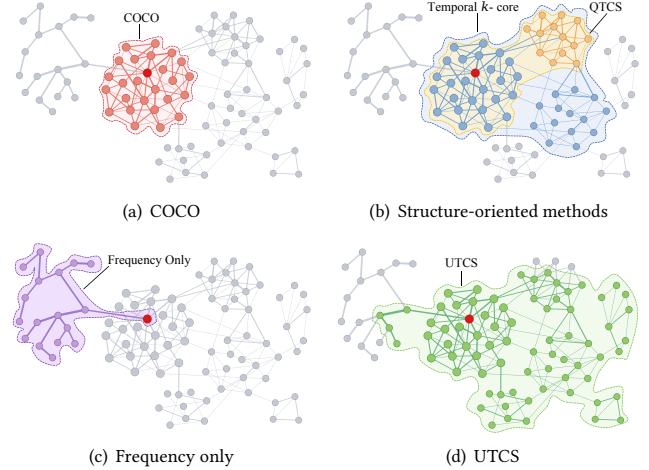


Figure 21: Case study on DBLP

author is red, and edge thickness represents interaction frequency. Figure 21(b) shows that temporal k -core returns a broad community, while QTCS selects a smaller query-centered region within it. Nevertheless, both include collaboration groups outside the densely interacting center, since neither directly evaluates whether internal collaborations are occasional or repeated. The frequency-only result in Figure 21(c) follows several thick edges into a branching peripheral region, but its selected authors have weak mutual connectivity. UTCS in Figure 21(d) returns an even broader community containing multiple peripheral groups. In contrast, COCO in Figure 21(a) focuses on the dense central group, where the authors are mutually connected and many internal edges represent repeated collaborations. It therefore excludes both the peripheral authors retained by structure-oriented methods and the weakly connected branch selected using interaction frequency alone. COCO thus identifies a more cohesive and active community than the baselines.

7 CONCLUSION

This paper addresses the problem of time-interval-based community search. We propose COCO, a cohesiveness-aware learning framework with three phases: pre-training, fine-tuning, and searching. Its adaptive query processing mechanism uses interval-specific coreness predictions from HM-Index to construct structural supervision and adaptation context for AT-GNN. The predicted coreness serves as a structural prior, enabling COCO to identify communities with strong structural cohesiveness and frequent interactions. COCO avoids recomputing k -core or retraining the entire model for each query. Experiments on real-world datasets demonstrate COCO’s community quality and computational efficiency.

REFERENCES

- [1] Ali Behrouz, Farnoosh Hashemi, and Laks VS Lakshmanan. 2022. FirmTruss community search in multilayer networks. *Proceedings of the VLDB Endowment* 16, 3 (2022), 505–518.
- [2] Xueqin Chang, Qing Liu, Yunjun Gao, Baihua Zheng, Yi Cai, and Qing Li. 2025. The most influenced community search on social networks. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2601–2614.
- [3] DBLP. 2024. <https://dblp.uni-trier.de/xml/>
- [4] Shuheng Fang, Kangfei Zhao, Yu Rong, Zhixun Li, and Jeffrey Xu Yu. 2024. Inductive attributed community search: to learn communities across graphs. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2576–2589.
- [5] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29 (2020), 353–392.
- [6] Yixiang Fang, Zhongran Wang, Reynold Cheng, Hongzhi Wang, and Jiafeng Hu. 2018. Effective and efficient community search over large directed graphs. *IEEE Transactions on Knowledge and Data Engineering* 31, 11 (2018), 2093–2107.
- [7] Edoardo Galimberti, Alain Barrat, Francesco Bonchi, Ciro Cattuto, and Francesco Gullo. 2018. Mining (maximal) span-cores from temporal networks. In *Proceedings of the 27th ACM international Conference on Information and Knowledge Management*. 107–116.
- [8] Edoardo Galimberti, Martino Ciaperoni, Alain Barrat, Francesco Bonchi, Ciro Cattuto, and Francesco Gullo. 2020. Span-core decomposition for temporal networks: Algorithms and applications. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 15, 1 (2020), 1–44.
- [9] Farnoosh Hashemi, Ali Behrouz, and Milad Rezaei Hajidehi. 2023. Cs-tgn: Community search via temporal graph neural networks. In *Companion Proceedings of the ACM Web Conference 2023*. 1196–1203.
- [10] Xin Huang, Laks VS Lakshmanan, and Jianliang Xu. 2019. *Community search over big graphs*. Morgan & Claypool Publishers.
- [11] Yuli Jiang, Yu Rong, Hong Cheng, Xin Huang, Kangfei Zhao, and Junzhou Huang. 2022. Query driven-graph neural networks for community search: From non-attributed, attributed, to interactive attributed. *Proceedings of the VLDB Endowment* 15, 6 (2022), 1243–1255.
- [12] Zongli Jiang, Yirui Tan, Guoxin Chen, Fangda Guo, Jinli Zhang, and Xiaolu Bai. 2024. GNN-Based Persistent K-core Community Search in Temporal Graphs. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 569–578.
- [13] Wissam Khaoioud, Marina Barsky, Venkatesh Srinivasan, and Alex Thomo. 2015. K-core decomposition of large networks on a single PC. *Proceedings of the VLDB Endowment* 9, 1 (2015), 13–23.
- [14] Junghoon Kim, Siqiang Luo, Gao Cong, and Wenyuan Yu. 2022. DMCS: Density Modularity Based Community Search. In *Proceedings of the 2022 International Conference on Management of Data*. 889–903. <https://doi.org/10.1145/3514221.3526137>
- [15] Jon M Kleinberg. 2000. Navigation in a small world. *Nature* 406, 6798 (2000), 845–845.
- [16] Ling Li, Siqiang Luo, Yuhai Zhao, Caihua Shan, Zhengkui Wang, and Lu Qin. 2023. COCLEP: Contrastive learning-based semi-supervised community search. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2483–2495.
- [17] Ling Li, Yuhai Zhao, Yuan Li, Fazal Wahab, and Zhengkui Wang. 2022. The most active community search in large temporal graphs. *Knowledge-Based Systems* 250 (2022), 109101.
- [18] Rong-Hua Li, Jiao Su, Lu Qin, Jeffrey Xu Yu, and Qiangqiang Dai. 2018. Persistent community search in temporal networks. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 797–808.
- [19] Yuan Li, Jinsheng Liu, Huiqun Zhao, Jing Sun, Yuhai Zhao, and Guoren Wang. 2021. Efficient continual cohesive subgraph search in large temporal graphs. *World Wide Web* 24 (2021), 1483–1509.
- [20] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, Chunxue Zhu, Hongchao Qin, Hai Jin, and Tao Jia. 2024. QTCs: Efficient Query-Centered Temporal Community Search. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1187–1199.
- [21] Qing Liu, Minjun Zhao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2020. Truss-based community search over large directed graphs. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2183–2197.
- [22] Qing Liu, Yifan Zhu, Minjun Zhao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2020. VAC: vertex-centric attributed community search. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 937–948.
- [23] Chengyang Luo, Chuanyu Liu, Qing Liu, Jianliang Xu, and Yunjun Gao. 2026. COCO: A Cohesiveness-aware Learning Framework for Community Search over Temporal Graphs (Technical Report). <https://github.com/ZJU-DAILY/COCO/blob/main/COCO-Technical%20Report.pdf>
- [24] Chengyang Luo, Qing Liu, Yunjun Gao, and Jianliang Xu. 2025. Synergetic Community Search over Large Multilayer Graphs. *Proceedings of the VLDB Endowment* 18, 5 (2025), 1412–1424.
- [25] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [26] Fragkiskos D Malliaros, Christos Giatsidis, Apostolos N Papadopoulos, and Michalis Vazirgiannis. 2020. The core decomposition of networks: Theory, algorithms and applications. *The VLDB Journal* 29, 1 (2020), 61–92.
- [27] Mark EJ Newman and Michelle Girvan. 2004. Finding and evaluating community structure in networks. *Physical Review E* 69, 2 (2004), 026113. <https://doi.org/10.1103/PhysRevE.69.026113>
- [28] Hongchao Qin, Rong-Hua Li, Guoren Wang, Lu Qin, Yurong Cheng, and Ye Yuan. 2019. Mining periodic cliques in temporal networks. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 1130–1141.
- [29] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, Lu Qin, and Zhiwei Zhang. 2022. Mining bursting core in large temporal graphs. *Proceedings of the VLDB Endowment* (2022).
- [30] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, Weihua Yang, and Lu Qin. 2020. Periodic communities mining in temporal networks: Concepts and algorithms. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2020), 3927–3945.
- [31] Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 815–823.
- [32] Qing Sima, Jianke Yu, Xiaoyang Wang, Wenjie Zhang, Ying Zhang, and Xuemin Lin. 2025. Deep Overlapping Community Search via Subspace Embedding. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–26.
- [33] Longxu Sun, Xin Huang, Zheng Wu, and Jianliang Xu. 2024. Efficient Cross-layer Community Search in Large Multilayer Graphs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2959–2971.
- [34] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- [35] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Efficient unsupervised community search with pre-trained graph transformer. *Proceedings of the VLDB Endowment* 17, 9 (2024), 2227–2240.
- [36] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Neural attributed community search at billion scale. *Proceedings of the ACM on Management of Data* 1, 4 (2024), 1–25.
- [37] Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, Lu Qin, and Yuting Zhang. 2021. Efficient and effective community search on large-scale bipartite graphs. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 85–96.
- [38] Kai Wang, Wenjie Zhang, Ying Zhang, Lu Qin, and Yuting Zhang. 2021. Discovering significant communities on bipartite graphs: An index-based approach. *IEEE Transactions on Knowledge and Data Engineering* 35, 3 (2021), 2471–2485.
- [39] Yuxiang Wang, Xiaoxuan Gou, Xiaoliang Xu, Yuxia Geng, Xiangyu Ke, Tianxing Wu, Zhiyuan Yu, Runhui Chen, and Xiangying Wu. 2024. Scalable community search over large-scale graphs based on graph transformer. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1680–1690.
- [40] Yue Wang, Xun Jian, Zhenhua Yang, and Jia Li. 2017. Query optimal k-plex based community in graphs. *Data Science and Engineering* 2 (2017), 257–273.
- [41] Yuxiang Wang, Shuzhan Ye, Xiaoliang Xu, Yuxia Geng, Zhenghe Zhao, Xiangyu Ke, and Tianxing Wu. 2024. Scalable community search with accuracy guarantee on attributed graphs. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2737–2750.
- [42] Junyong Yang, Ming Zhong, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2023. Scalable time-range k-core query on temporal graphs (full version). *arXiv preprint arXiv:2301.03770* (2023).
- [43] Junyong Yang, Ming Zhong, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2024. Evolution Forest Index: Towards Optimal Temporal k-Core Component Search via Time-Topology Isomorphic Computation. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2840–2853.
- [44] Michael Yu, Dong Wen, Lu Qin, Ying Zhang, Wenjie Zhang, and Xuemin Lin. 2021. On querying historical k-cores. *Proceedings of the VLDB Endowment* (2021).
- [45] Yue Zhang, Yankai Chen, Yingli Zhou, Yucan Guo, Xiaolin Han, and Chenhao Ma. 2025. UTCS: Effective Unsupervised Temporal Community Search with Pre-training of Temporal Dynamics and Subgraph Knowledge. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2647–2651.
- [46] Yifei Zhang, Longlong Lin, Pingpeng Yuan, and Hai Jin. 2022. Significant engagement community search on temporal networks. In *International Conference on Database Systems for Advanced Applications*. Springer, 250–258.
- [47] Ming Zhong, Junyong Yang, Yuanyuan Zhu, Tiejun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2024. A Unified and Scalable Algorithm Framework of User-Defined Temporal (k, X)-Core Query. *IEEE Transactions on Knowledge and Data Engineering* (2024).

A DISCUSSION OF LEMMA 4.1

Table 5: Error propagation through HM-Index

Dataset	$\max \beta_k$	$\max \eta_k$	$\bar{\epsilon}_h$	$\bar{U}_h$	Gap
CM	0.5683	0.2217	0.8912	1.2765	0.3859
MO	0.6714	0.3448	0.8016	1.9911	1.1903
SU	0.6821	0.3355	0.6438	2.0257	1.3824
WT	0.7529	0.3942	0.9634	3.4826	2.5190
YT	0.6836	0.3203	1.4071	2.2708	0.8635
FL	0.6587	0.3820	1.9959	2.7762	0.7816
SO	0.6722	0.3434	0.6697	2.1693	1.5008
WP	0.6605	0.3081	2.1496	2.8714	0.7227

Contraction assumption. Lemma 4.1 treats $\beta < 1$ as a verifiable property of the hierarchical refinement process rather than a first-principles guarantee. The parent prediction enters Eq. (2) through $\mathbf{W}_0 \hat{c}$, while each child model uses interval-specific features to refine the inherited estimate. This residual design makes contraction plausible but does not guarantee it. The evaluation below estimates a separate nonnegative slope β_k at each depth from consecutive prediction errors. Table 5 reports $\max_k \beta_k$ rather than one pooled slope. The values range from 0.568 to 0.752 across the eight datasets and remain below 1.

Overfitting at deeper nodes. Every HM-Index node is trained with the same number of vertex batches of size n_b and the same λ training epochs, and each sampled vertex is paired with a randomly selected subinterval. Thus, the number of sampled vertex-interval pairs does not decrease with depth. The dimension of $\mathbf{s}_2$ is bounded by p , while the maximum dimension of $\mathbf{s}_1$ decreases with $|T_N|$. The model capacity therefore does not increase and generally decreases with depth, which reduces the risk of overfitting. Although the local signal may become less informative at deeper levels, the additive error term η accounts for the unresolved component in residual refinement.

Tightness of the bound. The term $\eta/(1 - \beta)$ is a conservative worst-case bound obtained by taking $\eta = \sup_j \eta_j$. It need not be attained by the actual error sequence. Retaining the approximation error at each level gives the tighter finite-depth form

$$\epsilon_h \leq \beta^h \epsilon_0 + \sum_{k=0}^{h-1} \beta^{h-1-k} \eta_k.$$

For each depth, the evaluation below sets η_k to the largest positive residual after fitting β_k . Table 5 reports that the largest η_k values range from 0.221 to 0.394. The mean final errors range from 0.643 to 2.149, while the corresponding uniform bounds range from 1.276 to 3.482. Their mean gaps range from 0.385 to 2.519, and every evaluated pair has a nonnegative gap. These results are consistent with the bound and show that it is conservative.

Empirical Evaluation of Error Propagation. For every vertex-interval pair i used in the evaluation, we record the absolute prediction error $\epsilon_{i,k}$ after each visited depth k . At each depth, we fit $\epsilon_{i,k} = \beta_k \epsilon_{i,k-1} + r_{i,k}$ with $\beta_k \geq 0$ and set $\eta_k = \max_i \max\{0, r_{i,k}\}$. This choice makes the resulting recurrence valid for every evaluated pair. We then use $\beta = \max_k \beta_k$ and $\eta = \max_k \eta_k$ to compute the uniform bound $U_{i,h}$ in Lemma 4.1. Table 5 reports $\max_k \beta_k$, $\max_k \eta_k$,

the mean observed final error, the mean uniform bound, and their mean gap.

B PROOF OF LEMMA 4.2

PROOF OF LEMMA 4.2. Let $\hat{a}_k = \frac{1}{\sqrt{T}} \sum_{t=0}^{T-1} a_t e^{-2\pi i k t/T}$ denote the k -th coefficient of the unitary DFT of $\mathbf{a}$, and similarly for $\hat{b}_k$. Since the DFT is linear, $\hat{a}_k - \hat{b}_k$ is the k -th DFT coefficient of the difference sequence $\mathbf{a} - \mathbf{b}$.

By the reverse triangle inequality for complex modulus, we have $||\hat{a}_k| - |\hat{b}_k|| \leq |\hat{a}_k - \hat{b}_k|$ for each frequency index k . Squaring and summing over the first d components yields $\|\phi_d(\mathbf{a}) - \phi_d(\mathbf{b})\|_2^2 = \sum_{k=0}^{d-1} (|\hat{a}_k| - |\hat{b}_k|)^2 \leq \sum_{k=0}^{d-1} |\hat{a}_k - \hat{b}_k|^2$.

Since $d \leq T$, the partial sum is bounded by the full sum: $\sum_{k=0}^{d-1} |\hat{a}_k - \hat{b}_k|^2 \leq \sum_{k=0}^{T-1} |\hat{a}_k - \hat{b}_k|^2$. By Parseval’s identity for the unitary DFT, this equals $\|\mathbf{a} - \mathbf{b}\|_2^2 = \sum_{t=0}^{T-1} (a_t - b_t)^2$. Because $\mathbf{a}, \mathbf{b} \in \{0, 1\}^T$, each term $(a_t - b_t)^2 \in \{0, 1\}$, and exactly m terms are nonzero by assumption. Therefore $\|\mathbf{a} - \mathbf{b}\|_2^2 = m$.

Combining the above, we obtain $\|\phi_d(\mathbf{a}) - \phi_d(\mathbf{b})\|_2^2 \leq m$, which gives $\|\phi_d(\mathbf{a}) - \phi_d(\mathbf{b})\|_2 \leq \sqrt{m}$. $\square$

C PROOF OF LEMMA 4.3

PROOF. For the fixed query context and ReLU activation region in the lemma, let $\mathbf{J}_\Phi$ be the Jacobian of the fusion module with respect to its projected vertex input, and let $\mathbf{D}_\sigma$ be the diagonal derivative matrix of the ReLU. The chain rule gives $\frac{\partial \text{ADP}_Q}{\partial \mathbf{h}} = \mathbf{D}_\sigma \mathbf{W}_{\text{up}} \mathbf{J}_\Phi \mathbf{W}_{\text{down}_1}$. The rank of a matrix product cannot exceed the rank of any factor. Because the signal passes through both the adapter bottleneck and the attention-head representation, the Jacobian rank is at most $\min\{\dim_{\text{ADP}}, \dim_{\text{head}}\} = \xi$.

For exact realization, consider an affine correction $\Delta_Q(\mathbf{h}) = \mathbf{A}_Q \mathbf{h} + \mathbf{b}_Q$ whose output remains componentwise nonnegative in the fixed region. Augment the input as $\tilde{\mathbf{h}} = [\mathbf{h}^\top \ 1]^\top$. If $r = \text{rank}([\mathbf{A}_Q \ \mathbf{b}_Q]) \leq \xi$, then $[\mathbf{A}_Q \ \mathbf{b}_Q] = \mathbf{U}\mathbf{V}^\top$ through an r -dimensional space. Write $\mathbf{V}^\top = [\mathbf{V}_h^\top \ \mathbf{v}_0]$. Embed $\mathbf{V}_h^\top$ and $\mathbf{U}$ into $\mathbf{W}_{\text{down}_1}$ and $\mathbf{W}_{\text{up}}$, respectively, with zero padding when needed. Since $\mathbf{s}_Q$ is fixed and nonzero for a valid query interval, choose $\mathbf{W}_{\text{down}_2}$ such that $\mathbf{W}_{\text{down}_2} \mathbf{s}_Q = \mathbf{v}_0$. Under the linear/identity fusion configuration, these branches produce $\mathbf{V}_h^\top \mathbf{h} + \mathbf{v}_0$. The pre-activation output is therefore $\mathbf{U}(\mathbf{V}_h^\top \mathbf{h} + \mathbf{v}_0) = \mathbf{A}_Q \mathbf{h} + \mathbf{b}_Q$. Its componentwise nonnegativity makes the ReLU an identity on this region, proving exact realization. $\square$

D MORE EXPERIMENTAL RESULTS

Community Tendency of the Datasets. For each query interval, we compute the global clustering coefficient (GCC). We form a temporal null model by shuffling timestamps among temporal edge events while retaining their endpoints, preserving full-lifespan interactions but removing temporal localization. Table 6 reports averages over 100 query intervals; Null denotes the shuffled-timestamp baseline, and Ratio is GCC divided by Null. Across the eight datasets, GCC ranges from 0.0025 to 0.1512 and is 3.1–27.0 times the null value. These values indicate temporal clustering beyond the null expectation and meaningful community structure.

Table 6: Community tendency of the datasets

Dataset	GCC	Null	Ratio
CollegeMsg	0.0637	0.0146	4.4
MathOverflow	0.1512	0.0220	6.8
Superuser	0.0219	0.0061	3.6
WikiTalk	0.0209	7.8×10^{-4}	27.0
Youtube	0.0025	1.3×10^{-4}	19.5
Flickr	0.0475	0.0127	3.8
StackOverflow	0.0169	0.0055	3.1
Wikipedia	0.0792	0.0112	7.1

Exp-14: Progressive Graph Randomization. We progressively rewire the temporal edges of all eight datasets from 0% to 100%. Each swap uses two edges with the same timestamp and exchanges their endpoints while rejecting self-loops and duplicate edges. This preserves every vertex’s temporal degree at each timestamp while weakening higher-order structure. We rebuild COCO for every randomized graph and use the same queries at all randomization levels. Figure 22 shows consistent changes across the eight datasets. At full randomization, TD decreases by 52.38%–58.16%, TM decreases by 77.33%–83.64%, and TC increases by 18.31%–37.80%. This degradation shows that the community structure identified by COCO depends on clustering in the input graphs.

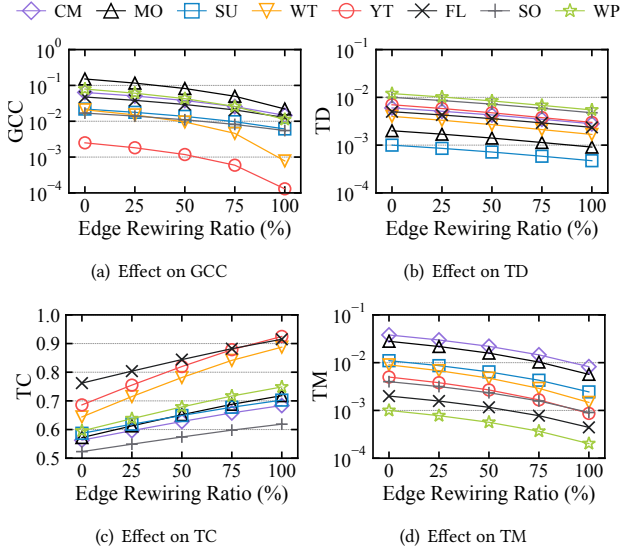


Figure 22: Effect of graph randomization

Additional TM Results for the Parameter Analysis. Figure 23 reports TM for the four parameters studied in Exp-8–Exp-11. The trends agree with the corresponding TD and TC results in the main paper.

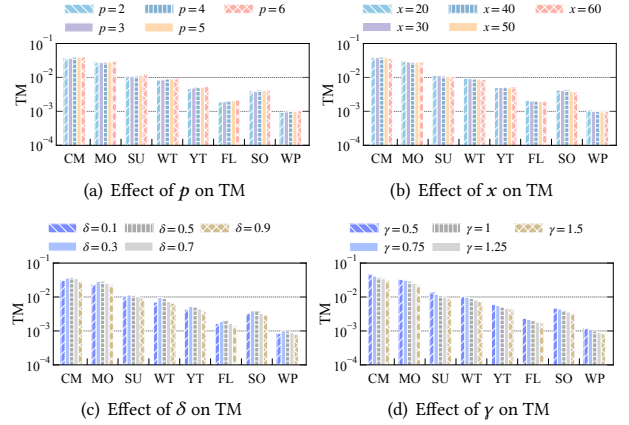


Figure 23: Effect of model parameters on temporal modularity

Exp-15: Effect of $|V_Q|$. Figure 24 reports TM as the number of query vertices varies. As $|V_Q|$ increases, TM decreases slightly across all datasets. Larger query sets typically require more intermediate vertices to maintain structural connectivity and cohesiveness, expanding community size. The resulting communities have lower internal interaction density, which explains the small decrease in TM. Despite this, COCO maintains stable performance across query scales, demonstrating its robustness in multi-vertex community search.

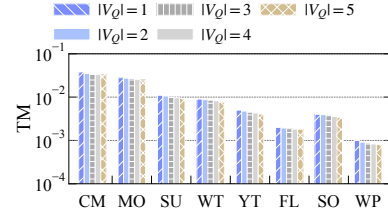


Figure 24: Effect of $|V_Q|$ on temporal modularity

Exp-16: Effect of # of Fine-tuning Steps. We explore the influence of the number of fine-tuning steps on COCO. Specifically, we test 5, 10, 15, 20, and 25 steps. Figure 25 shows that TM increases as more steps are used. The largest gains occur between 10 and 15 steps, and the improvement becomes smaller thereafter. This suggests that COCO benefits from sufficient adaptation (≥ 15 steps), while excessive fine-tuning yields limited additional improvement.

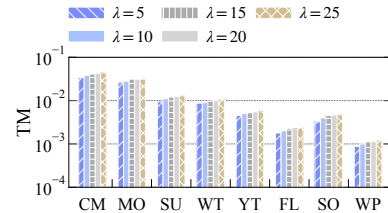


Figure 25: Effect of fine-tuning steps on temporal modularity

Table 7: Community quality comparison for the worst 5% of queries

Datasets	CM			MO			SU			WT			YT			FL			SO			WP			Avg +/-		
Metrics	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM	TD	TC	TM
COCO	0.0042	0.5837	0.0194	0.0013	0.5882	0.0146	6.2×10^{-4}	0.6105	0.0053	0.0024	0.6593	0.0042	0.0041	0.7118	0.0023	0.0032	0.7814	8.3×10^{-4}	0.0064	0.5457	0.0024	0.0072	0.6109	4.2×10^{-4}	-	-	-
Temporal k-core	0.0023	0.7785	0.0094	2.7×10^{-4}	0.8173	0.0042	3.8×10^{-5}	0.7854	3.4×10^{-4}	6.9×10^{-4}	0.8956	8.9×10^{-4}	2.9×10^{-4}	0.8325	1.6×10^{-4}	3.5×10^{-5}	0.9034	1.8×10^{-5}	0.0012	0.8126	3.9×10^{-4}	0.0023	0.9015	1.5×10^{-4}	-77.0%	+33.3%	-79.5%
QTCS	0.0032	0.7415	0.0124	5.8×10^{-4}	0.7553	0.0072	2.8×10^{-4}	0.7664	0.0023	9.8×10^{-4}	0.8315	0.0012	0.0012	0.9274	4.8×10^{-4}	0.0012	0.9283	3.2×10^{-4}	0.0023	0.8665	5.3×10^{-4}	OOM	OOM	OOM	-54.8%	+30.8%	-60.1%
TransZero	0.0012	0.9174	0.0023	7.3×10^{-5}	0.9483	3.8×10^{-4}	2.8×10^{-5}	0.9643	1.9×10^{-4}	3.2×10^{-5}	0.9364	4.3×10^{-5}	1.3×10^{-4}	0.9843	4.2×10^{-5}	8.3×10^{-5}	0.9942	8.4×10^{-6}	1.9×10^{-5}	0.9903	4.2×10^{-6}	OOM	OOM	OOM	-93.6%	+52.3%	-97.2%
CS-Former	2.4×10^{-4}	0.9543	-0.0152	9.3×10^{-5}	0.9774	1.3×10^{-4}	2.3×10^{-6}	0.9853	-1.2×10^{-3}	5.3×10^{-5}	0.9704	4.2×10^{-5}	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-95.7%	+59.6%	-124.3%
IACS	0.0013	0.9764	-0.0062	6.3×10^{-5}	0.9114	4.8×10^{-4}	2.8×10^{-5}	0.9963	-1.4×10^{-3}	1.6×10^{-4}	0.7934	1.2×10^{-4}	5.3×10^{-8}	0.9394	1.7×10^{-7}	2.3×10^{-8}	0.9943	4.3×10^{-8}	2.8×10^{-7}	0.9474	1.9×10^{-7}	OOM	OOM	OOM	-93.5%	+48.4%	-107.2%
CS-TGN	8.3×10^{-4}	0.9534	3.9×10^{-4}	1.7×10^{-5}	0.8043	4.3×10^{-4}	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-88.7%	+50.1%	-97.6%
UTCS	8.3×10^{-4}	0.7904	0.0042	7.3×10^{-4}	0.7654	0.0093	1.4×10^{-4}	0.7534	0.0012	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM	-63.0%	+29.7%	-65.4%

* A community is considered to be of higher quality when it exhibits a **higher TD**, a **lower TC**, and a **higher TM**.

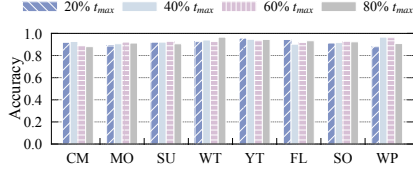


Figure 26: Effect of $|T_Q|$ on prediction accuracy

Exp-17: Effect of $|T_Q|$ on Prediction Accuracy of HM-Index. In this experiment, we evaluate the prediction accuracy of HM-Index by predicting vertex coreness across progressive time intervals (20%, 40%, 60%, and 80% of $t_{\max}$). Results shown in Figure 26 demonstrate consistent performance above 90% accuracy across all datasets and interval lengths. This robustness stems from HM-Index’s hierarchical architecture. COCO initiates its search from the root node, traverses through the HM-Index to identify nodes encompassing the query time interval. The models along the search path collectively refine coreness predictions.

Exp-18: Effect of p on Prediction Accuracy of HM-Index. The effect of the partition coefficient p of HM-Index is shown in Figure 27. As observed in the figure, the prediction accuracy shows a consistent upward trend as p increases across all datasets. This improvement is primarily because a larger partition coefficient leads to more branches in the hierarchical tree, allowing for a finer-grained division of the time intervals. Such detailed partitioning enables the lightweight regression models at each node to better capture subtle temporal dynamics and structural properties within specific sub-intervals, thereby refining the coreness estimation.

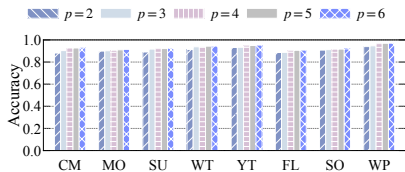


Figure 27: Effect of p on prediction accuracy

Exp-19: Effect of x on Prediction Accuracy of HM-Index. The effect of the leaf node threshold x , used when constructing the HM-Index, on prediction accuracy is shown in Figure 28. As shown in the figure, prediction accuracy slightly declines as x increases

from 20 to 60 across all datasets. A smaller x yields finer temporal partitions in the HM-Index leaf nodes, allowing regression models to better capture short-term dynamics and structural variations in vertex coreness. In contrast, larger x leads to coarser intervals that may miss subtle temporal patterns, causing a minor accuracy drop. Nevertheless, accuracy remains above 90% for all x values.

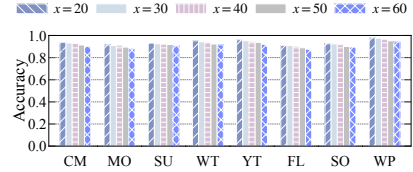


Figure 28: Effect of x on prediction accuracy

Exp-20: Lower-Tail Quality under Adverse Queries. We examine the lower-tail behavior of all methods using the same 100 queries per dataset as Exp-1. For each dataset, we identify the five queries on which COCO performs worst, corresponding to the worst 5%, and report the mean results of all methods and metrics on this fixed query set.

Table 7 compares all methods under this lower-tail setting. COCO consistently achieves higher TD, lower TC, and higher TM than the baselines. Compared with QTCS, the average differences are 54.8% in TD, 30.8% in TC, and 60.1% in TM, exceeding the corresponding differences of 43.22%, 20.59%, and 48.58% in Table 3. Compared with the average COCO results, TD decreases by 27.78%–43.64%, TM decreases by 42.86%–61.82%. These results show that COCO maintains a larger advantage over the baselines on adverse queries.

These lower-tail results do not establish a per-query quality guarantee. Potential failure cases include query vertices near weak or overlapping community boundaries and query intervals with sparse or irregular interactions. In these settings, small errors in predicted coreness or temporal representations can alter the ranking of candidates with similar scores, which may change the returned community. The output should therefore be interpreted cautiously when the input provides little structural or temporal community signal. Exact methods suit applications requiring hard combinatorial guarantees, whereas learning-based methods such as COCO better suit large temporal graphs where exact per-query computation is impractical and empirical community quality is prioritized.